

Oracle®

Universal Connection Pool Developer's Guide



19c
E96473-05
August 2025

ORACLE®

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

Contents

Preface

Audience	i
Documentation Accessibility	i
Related Documents	i
Conventions	i

Changes in This Release for Oracle Universal Connection Pool Developer's Guide

Changes in Oracle Database 19c	i
--------------------------------	---

1 Introduction to UCP

1.1 Overview of Connection Pool	1
1.2 Benefits of Using a Connection Pool	1
1.3 Overview of Universal Connection Pool	2
1.3.1 Conceptual Architecture	2
1.3.2 Connection Pool Properties	3
1.3.3 Connection Pool Manager	3
1.3.4 High Availability and Performance Scenarios	3
1.3.5 Support for Java library for Reactive Streams Ingestion	4

2 Getting Started

2.1 Requirements for using UCP	1
2.2 Basic Connection Steps in UCP	1
2.2.1 Authentication in UCP	2
2.2.2 Authentication Using IAM Database Access Tokens in Oracle Cloud Infrastructure	2
2.3 UCP API Overview	3
2.4 Basic Connection Example Using UCP	3

3 Getting Database Connections in UCP

3.1	About Borrowing Connections from UCP	1
3.1.1	Overview of Borrowing Connections from UCP	1
3.1.2	Using the Pool-Enabled Data Source	2
3.1.3	Using the Pool-Enabled XA Data Source	3
3.1.4	Setting Connection Properties	4
3.1.5	Using JNDI to Borrow a Connection	4
3.1.6	About Connection Initialization Callback	5
3.1.6.1	Overview of Connection Initialization Callback	5
3.1.6.2	Creating an Initialization Callback	5
3.1.6.3	Registering an Initialization Callback	6
3.1.6.4	Removing or Unregistering an Initialization Callback	6
3.2	Setting Connection Pool Properties for UCP	6
3.3	Overview of Validating Connections in UCP	7
3.3.1	Validating When Borrowing	7
3.3.2	Minimizing Connection Validation with <code>setSecondsToTrustIdleConnection()</code> Method	8
3.3.3	Checking If a Connection Is Valid	8
3.4	Returning Borrowed Connections to UCP	9
3.5	Removing Connections from UCP	10
3.6	UCP Integration with Third-Party Products	10

4 Optimizing Universal Connection Pool Behavior

4.1	Optimizing Connection Pools	1
4.2	About Controlling the Pool Size in UCP	2
4.2.1	Setting the Initial Pool Size	2
4.2.2	Setting the Minimum Pool Size	2
4.2.3	Setting the Maximum Pool Size	2
4.3	Real-World Performance Considerations with Respect to Connection Pool Sizes	3
4.4	Stale Connections in UCP	4
4.4.1	What is Connection Reuse?	4
4.4.1.1	Setting the Maximum Connection Reuse Time	4
4.4.1.2	Setting the Maximum Connection Reuse Count	5
4.4.2	Setting the Connection Validation Timeout	5
4.4.3	Setting the Abandon Connection Timeout	5
4.4.4	Setting the Time-To-Live Connection Timeout	6
4.4.5	Setting the Connection Wait Timeout	6
4.4.6	Setting the Inactive Connection Timeout	7
4.4.7	Setting the Query Timeout	7
4.4.8	Setting the Timeout Check Interval	7
4.5	About Harvesting Connections in UCP	8

4.5.1	Overview of Harvesting Connections in UCP	8
4.5.2	Setting a Connection to Harvestable	8
4.5.3	Setting the Harvest Trigger Count	9
4.5.4	Setting the Harvest Maximum Count	9
4.6	About Caching SQL Statements in UCP	10
4.6.1	Overview of Statement Caching in UCP	10
4.6.2	Enabling Statement Caching in UCP	10

5 Labeling Connections in UCP

5.1	Overview of Labeling Connections in UCP	1
5.2	Implementation of a Labeling Callback in UCP	1
5.2.1	When to Use a Labeling Callback in UCP	2
5.2.2	Creating a Labeling Callback in UCP	2
5.2.2.1	Example of Labeling Callback in UCP	3
5.2.3	Registering a Labeling Callback in UCP	4
5.2.4	Removing a Labeling Callback in UCP	4
5.3	Integration of UCP with DRCP	4
5.4	Applying Connection Labels in UCP	5
5.5	Borrowing Labeled Connections from UCP	6
5.6	Checking Unmatched Labels in UCP	6
5.7	Removing a Connection Label in UCP	6

6 Controlling Reclaimable Connection Behavior

6.1	AbandonedConnectionTimeoutCallback Interface	1
6.2	TimeToLiveConnectionTimeoutCallback Interface	1

7 Using the Connection Pool Manager

7.1	Overview of Using the UCP Manager	1
7.1.1	About Connection Pool Manager	1
7.1.2	Creating a Connection Pool Manager for UCP	1
7.1.3	Life Cycle States of a Connection	1
7.1.3.1	Creating a Connection Pool	2
7.1.3.2	Starting a Connection Pool	3
7.1.3.3	Stopping a Connection Pool	3
7.1.3.4	Destroying a Connection Pool	3
7.1.4	Maintenance of Universal Connection Pool	3
7.1.4.1	Refreshing a Connection Pool	4
7.1.4.2	Recycling a Connection Pool	4
7.1.4.3	Purging a Connection Pool	4

7.2	Overview of JMX-Based Management in UCP	5
7.2.1	UniversalConnectionPoolManagerMBean	5
7.2.2	UniversalConnectionPoolMBean	6

8 Shared Pool Support for Multitenant Data Sources

8.1	Overview of Shared Pool Support	1
8.2	Prerequisites for Supporting Shared Pool	4
8.3	Configuring the Shared Pool	5
8.4	UCP APIs for Shared Pool Support	6
8.5	Sample XML Configuration File for Shared Pool	7

9 Using Oracle RAC Features

9.1	Overview of Oracle RAC Features	1
9.2	About Fast Connection Failover	2
9.2.1	Overview of Fast Connection Failover	2
9.2.2	What is Fast Connection Failover?	4
9.2.2.1	What the Application Sees	4
9.2.2.2	How FCF Works	4
9.2.3	Fast Connection Failover Prerequisites	5
9.2.4	Example of Fast Connection Failover Configuration	5
9.2.5	Enabling Fast Connection Failover	6
9.2.6	What is ONS?	6
9.2.6.1	Overview of ONS Configuration File	7
9.2.6.2	Remote Configuration of ONS	9
9.2.6.3	Configuration of Client-Side ONS Daemon	10
9.2.7	Configuring the Connection URL	12
9.3	About Run-Time Connection Load Balancing	13
9.3.1	Overview of Run-Time Connection Load Balancing	13
9.3.2	Setting Up Run-Time Connection Load Balancing	14
9.4	About Connection Affinity	14
9.4.1	Overview of Connection Affinity	15
9.4.1.1	Transaction-Based Affinity	15
9.4.1.2	Web Session Affinity	15
9.4.1.3	Oracle RAC Data Affinity	16
9.4.2	Setting Up Connection Affinity	17
9.4.2.1	Creating a Connection Affinity Callback	17
9.4.2.2	Registering a Connection Affinity Callback	18
9.4.2.3	Removing a Connection Affinity Callback	18
9.4.2.4	Strict Affinity Mode	19
9.5	Global Data Services	19

9.5.1	Overview of Global Data Services	19
9.5.2	Configuring an Application for Using GDS	20

10 Ensuring Application Continuity

10.1	Overview of Ensuring Application Continuity with UCP	1
10.2	Configuring the Data Source for Application Continuity	1
10.3	Using Connection Labeling for Application Continuity	2
10.4	Using Connection Initialization Callback for Application Continuity	2

11 Shared Pool for Sharded Databases

11.1	Overview of UCP Shared Pool for Database Sharding	1
11.2	About Handling Connection Requests for a Sharded Database	2
11.2.1	About Building the Sharding Key	3
11.2.2	How to Checkout Connections from a Pool with a Sharding Key	4
11.2.3	About Checking Out Connections without Providing the Sharding Keys	5
11.2.4	About Connecting to the Shard Catalog or Co-ordinator for Multi Shard Queries	5
11.2.5	About Configuring the Number of Connections Per Shard	5
11.2.6	Pool Connection Selection Algorithm During Connection Checkout	6
11.2.7	Failover or Resharding Event Handling in UCP	6
11.3	Middle-Tier Routing Using UCP	6
11.4	UCP APIs for Database Sharding Support	7
11.5	UCP APIs for Middle-Tier Routing Support	8
11.6	UCP Sharding Example	9
11.7	Middle-Tier Routing with UCP Example	9

12 Diagnosing a Connection Pool

12.1	Pool Statistics	1
12.2	Dynamic Monitoring Service Metrics	1
12.3	About Viewing Oracle RAC Statistics	2
12.3.1	Fast Connection Failover Statistics	2
12.3.2	Run-Time Connection Load Balance Statistics	3
12.3.3	Connection Affinity Statistics	3
12.4	Overview of Logging in UCP	3
12.4.1	Using a Logging Properties File	3
12.4.2	Using UCP and JDK API	4
12.4.3	Enabling or Disabling Feature-Specific Logging at Runtime	4
12.4.4	About Using the Logging Properties File for Feature-Specific Logging	5
12.4.5	Supported Log Levels	5

A Error Codes Reference

A.1	General Structure of UCP Error Messages	A-1
A.2	Connection Pool Layer Error Messages	A-1
A.3	JDBC Data Sources and Dynamic Proxies Error Messages	A-5

Index

Preface

The Oracle Universal Connection Pool (UCP) is a full-featured connection pool for managing database connections. Java applications that are database-intensive, use the connection pool to improve performance and better utilize system resources.

The instructions in this guide detail how to use the UCP API and cover a wide range of use cases. The guide does not provide detailed information about using Oracle JDBC Drivers, Oracle Database, or SQL except as required to understand UCP.

Audience

This guide is primarily written for Application Developers and System Architects who want to learn how to use UCP to create and manage database connections for their Java applications. Users must be familiar with Java and JDBC to use this guide. Knowledge of Oracle Database concepts (such as Oracle RAC and ONS) is required when using some UCP features.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Related Documents

For more information about using Java with the Oracle Database, see the following documents in the Oracle Database documentation set:

- *Oracle Database JDBC Developer's Guide*
- *Oracle Database 2 Day + Java Developer's Guide*
- *Oracle Database Java Developer's Guide*

Conventions

The following text conventions are used in this document:

Convention	Meaning
boldface	Boldface type indicates graphical user interface elements associated with an action, or terms defined in text or the glossary.

Convention	Meaning
<i>italic</i>	Italic type indicates book titles, emphasis, or placeholder variables for which you supply particular values.
monospace	Monospace type indicates commands within a paragraph, URLs, code in examples, text that appears on the screen, or text that you enter.

Changes in This Release for Oracle Universal Connection Pool Developer's Guide

This preface contains:

- [Changes in Oracle Database 19c](#)

Changes in Oracle Database 19c

The following are changes in *Oracle Universal Connection Pool Developer's Guide* for Oracle Database 19c.

New Features

This section lists the new features in this release.

- Support for Reactive Streams Ingestion
See "[Support for Java library for Reactive Streams Ingestion](#)"
- Setting the Connection Validation Timeout
See "[Setting the Connection Validation Timeout](#)"
- Support for IAM Token-Based Authentication
See "[Authentication Using IAM Database Access Tokens in Oracle Cloud Infrastructure](#)"

1

Introduction to UCP

The following sections are included in this chapter:

- [Overview of Connection Pool](#)
- [Overview of Universal Connection Pool](#)

1.1 Overview of Connection Pool

A connection pool is a cache of database connection objects. The objects represent physical database connections that can be used by an application to connect to a database. At run time, the application requests a connection from the pool. If the pool contains a connection that can satisfy the request, it returns the connection to the application. If no connections are found, a new connection is created and returned to the application. The application uses the connection to perform some work on the database and then returns the object back to the pool. The connection is then available for the next connection request.

Connection pools promote the reuse of connection objects and reduce the number of times that connection objects are created. Connection pools significantly improve performance for database-intensive applications because creating connection objects is costly both in terms of time and resources. Tasks such as network communication, reading connection strings, authentication, transaction enlistment, and memory allocation all contribute to the amount of time and resources it takes to create a connection object. In addition, because the connections are already created, the application waits less time to get the connection.

Connection pools often provide properties that are used to optimize the performance of a pool. The properties control behaviors such as the minimum and maximum number of connections allowed in the pool or the amount of time a connection can remain idle before it is returned to the pool. The best configured connection pools balance quick response times with the memory spent maintaining connections in the pool. It is often necessary to try different settings until the best balance is achieved for a specific application.

1.2 Benefits of Using a Connection Pool

Applications that are database-intensive, generally benefit the most from connection pools. As a policy, applications should use a connection pool whenever database usage is known to affect application performance.

A connection pool provides the following benefits:

- Reduces the number of times new connection objects are created.
- Promotes connection object reuse.
- Quickens the process of getting a connection.
- Reduces the amount of effort required to manually manage connection objects.
- Minimizes the number of stale connections.
- Controls the amount of resources spent on maintaining connections.

1.3 Overview of Universal Connection Pool

UCP provides a connection pool implementation for caching JDBC connections. Java applications that are database-intensive use the connection pool to improve performance and better utilize system resources.

A UCP JDBC connection pool can use any JDBC driver to create physical connections that are then maintained by the pool. The pool can be configured and provides a full set of properties that are used to optimize pool behavior based on the performance and availability requirements of an application. For more advanced applications, UCP provides a pool manager that can be used to manage a pool instance.

The pool also leverages many high availability and performance features available through an Oracle Real Application Clusters (Oracle RAC) database. These features include Fast Connection Failover (FCF), Run-time connection Load Balancing (RLB), and Connection Affinity.

Note

Starting from Oracle Database 11g Release 2, FCF is also supported by Oracle Restart on a single instance database. Oracle Restart is also known as Oracle Grid Infrastructure for Independent Servers.

See Also

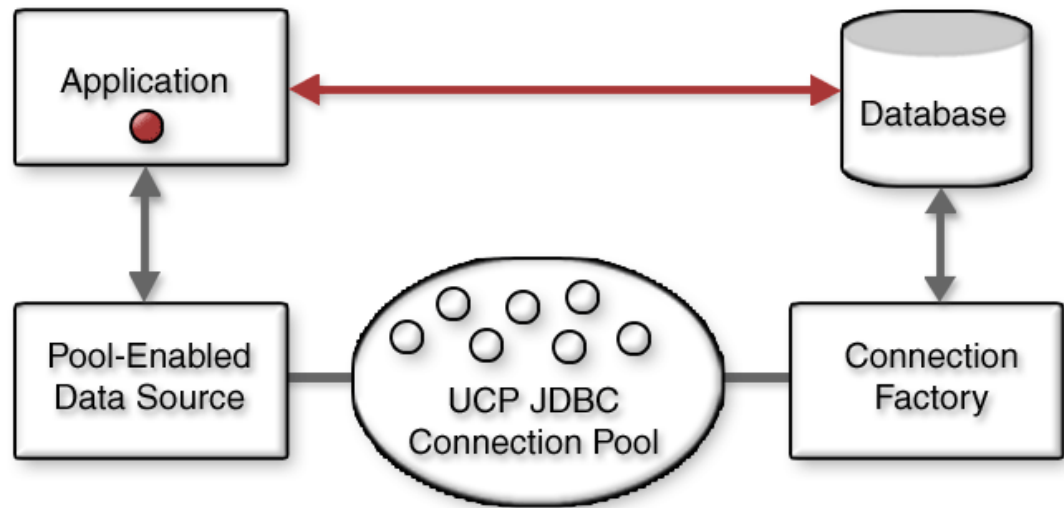
Oracle Database Administrator's Guide for more information about Oracle Restart.

1.3.1 Conceptual Architecture

Applications use a UCP pool-enabled data source to get connections from a UCP JDBC connection pool instance. The `PoolDataSource` data source is used for getting regular connections (`java.sql.Connection`), and the `PoolXADataSource` data source is used for getting XA (eXtended API) connections (`javax.sql.XAConnection`). The same pool features are included in both XA and non-XA UCP JDBC connection pools.

The pool-enabled data source relies on a connection factory class to create the physical connections that are maintained by the pool. An application can choose to use any factory class that is capable of creating `Connection` or `XAConnection` objects. The pool-enabled data sources provide a method for setting the connection factory class, as well as methods for setting the database URL and database credentials that are used by the factory class to connect to a database.

Applications borrow a connection handle from the pool to perform work on a database. Once the work is completed, the connection is closed and the connection handle is returned to pool and is available to be used again. The following figure shows the conceptual view of the interaction between an application and a UCP JDBC connection pool.

Figure 1-1 Conceptual View of a UCP JDBC Connection Pool**Related Topics**

- [Getting Database Connections in UCP](#)

1.3.2 Connection Pool Properties

UCP JDBC Connection pool properties are configured through methods available on the pool-enabled data source. The pool properties are used to control the pool size, handle stale connections, and make autonomous decisions about how long connections can remain borrowed before they are returned to the pool. The optimal settings for the pool properties depend on the application and hardware resources. Typically, there is a trade-off between the time it takes for an application to get a connection versus the amount of memory it takes to maintain a certain pool size. In many cases, experimentation is required to find the optimal balance to achieve the desired performance for a specific application.

Related Topics

- [Optimizing Universal Connection Pool Behavior](#)

1.3.3 Connection Pool Manager

UCP includes a connection pool manager that is used by applications that require administrative control over a connection pool. The manager is used to explicitly control the life cycle of a pool and to perform maintenance on a pool. The manager also provides the opportunity for an application to expose the pool and its manageability through an administrative console.

Related Topics

- [Using the Connection Pool Manager](#)

1.3.4 High Availability and Performance Scenarios

A UCP JDBC connection pool provides many features that are used to ensure high connection availability and performance. Many of these features, such as refreshing a pool or validating

connections, are generic and work across driver and database implementations. Some of these features, such as run-time connection load balancing, and connection affinity, require the use of an Oracle JDBC driver and an Oracle RAC database.

Related Topics

- [Using Oracle RAC Features](#)

1.3.5 Support for Java library for Reactive Streams Ingestion

Oracle Database Release 19c introduces a Java library that provides support for reactive streams ingestion, which enables customers to efficiently stream data into Oracle Database. The new Java library enables Java applications to continuously receive and ingest data from a large group of clients. Using the direct path load method of Oracle Database for inserting data, it makes the ingestion process nonblocking and extremely fast.

This Java library uses an extension of the existing UCP APIs, which enables the ingestion process to furnish several high-availability and scalability features of the database, like support for table partitions, Oracle RAC connection affinity, and sharding.

① See Also

Overview of Java library for Reactive Streams Ingestion

2

Getting Started

The following sections are included in this chapter:

- [Requirements for using UCP](#)
- [Basic Connection Steps in UCP](#)
- [UCP API Overview](#)
- [Basic Connection Example Using UCP](#)

2.1 Requirements for using UCP

UCP has the following design-time and run-time requirements:

- JRE 1.5 or higher
- A JDBC driver or a connection factory class capable of returning a `java.sql.Connection` and `javax.sql.XAConnection` object

Note

Oracle drivers from releases 10.1 or higher are supported. Advanced Oracle Database features, such as Oracle RAC and Fast Connection Failover, require the Oracle Notification Service library (`ons.jar`) that is included with the Oracle Client software.

- The `ucp.jar` library included in the classpath of the application
- The `ojdbc8.jar` library included in the classpath of the application

Note

Even if you use UCP with a third-party database and driver, you *must* use the Oracle `ojdbc8.jar` library because UCP has dependencies on this library.

- A database that supports SQL. Advanced features, such as Oracle RAC and Fast Connection Failover, require an Oracle Database.

2.2 Basic Connection Steps in UCP

UCP provides a pool-enabled data source that is used by applications to borrow connections from a UCP JDBC connection pool. A connection pool is not explicitly defined for the most basic use case. Instead, a default connection pool is implicitly created when the connection is borrowed.

The following steps describe how to get a connection from a UCP pool-enabled data source in order to access a database. The complete example is provided in [Example 2-1](#):

1. Use the UCP data source factory (`oracle.ucp.jdbc.PoolDataSourceFactory`) to get an instance of a pool-enabled data source using the `getPoolDataSource` method. The data source instance must be of the type `PoolDataSource`. For example:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
```

2. Set the connection properties that are required to get a physical connection to a database. These properties are set on the data source instance and include: the URL, the user name, and password to connect to the database and the connection factory used to get the physical connection. These properties are specific to a JDBC driver and database. For example:

```
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");  
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");  
pds.setUser("<user>");  
pds.setPassword("<password>");
```

3. Set any pool properties in order to override the connection pool's default behavior. the pool properties are set on the data source instance. For example:

```
pds.setInitialPoolSize(5);
```

4. Get a connection using the data source instance. The returned connection is a logical handle to a physical connection in the data source's connection pool. For example:

```
Connection conn = pds.getConnection();
```

5. Use the connection to perform some work on the database:

```
Statement stmt = conn.createStatement ();  
stmt.execute("SELECT * FROM foo");
```

6. Close the connection and return it to the pool.

```
conn.close();
```

2.2.1 Authentication in UCP

UCP provides transparent authentication, that is, the `PoolDataSource` behaves in the same way as the `OracleDataSource` while authenticating a connection.

UCP supports all the following authentication methods that the JDBC thin or the JDBC OCI driver suggests, and delegates any authentication action to the underlying driver:

- Authentication through passwords stored in Oracle Wallets
- Authentication using Kerberos
- Authentication through SSL certificates
- Authentication using Lightweight Directory Access Protocol (LDAP)

2.2.2 Authentication Using IAM Database Access Tokens in Oracle Cloud Infrastructure

In Oracle Database release 19.13 (19.13.0.0.1), the JDBC Thin drivers can access Oracle Autonomous Database on Shared Exadata Infrastructure, using a database access token

generated by the Identity and Access Management (IAM) Cloud Service. UCP supports this authentication type using the `PoolDataSource.setTokenSupplier(Supplier)` method.

See Also

Support for IAM Token-Based Authentication in Oracle Cloud Infrastructure

2.3 UCP API Overview

The following section provides a quick overview of the most commonly used packages of the UCP API.

See Also

Oracle Universal Connection Pool Java API Reference for complete details on the API.

oracle.ucp.jdbc

This package includes various interfaces and classes that are used by applications to work with JDBC connections and a connection pool. Among the interfaces found in this package, the `PoolDataSource` and `PoolXADataSource` data source interfaces are used by an application to get connections as well as get and set connection pool properties. Data source instances implementing these two interfaces automatically create a connection pool.

oracle.ucp.admin

This package includes interfaces for using a connection pool manager as well as MBeans that allow users to access connection pool and the connection pool manager operations and attributes using JMX operations. Among the interfaces, the `UniversalConnectionPoolManager` interface provides methods for creating and maintaining connection pool instances.

oracle.ucp

This package includes both required and optional callback interfaces that are used to implement connection pool features. For example, the `ConnectionAffinityCallback` interface is used to create a callback that enables or disables connection affinity and can also be used to customize connection affinity behavior. This package also contains statistics classes, UCP specific exception classes, and the logic to use the UCP directly, without using data sources.

2.4 Basic Connection Example Using UCP

The following example is a program that connects to a database to do some work and then exits. The example is simple and in some cases not very practical; however, it does demonstrate the basic steps required to get a connection from a UCP pooled-enabled data source in order to access a database.

Example 2-1 Basic Connection Example

```
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
import oracle.ucp.jdbc.PoolDataSourceFactory;
```

```
import oracle.ucp.jdbc.PoolDataSource;

public class BasicConnectionExample {
    public static void main(String args[]) throws SQLException {
        try
        {
            //Create pool-enabled data source instance.

            PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

            //set the connection properties on the data source.

            pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
            pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");
            pds.setUser("<user>");
            pds.setPassword("<password>");

            //Override any pool properties.

            pds.setInitialPoolSize(5);

            //Get a database connection from the datasource.

            Connection conn = pds.getConnection();

            System.out.println("\nConnection obtained from " +
                "UniversalConnectionPool\n");

            //do some work with the connection.
            Statement stmt = conn.createStatement();
            stmt.execute("select * from foo");

            //Close the Connection.

            conn.close();
            conn=null;

            System.out.println("Connection returned to the " +
                "UniversalConnectionPool\n");

        }
        catch(SQLException e)
        {
            System.out.println("BasicConnectionExample - " +
                "main()-SQLException occurred : "
                + e.getMessage());
        }
    }
}
```

3

Getting Database Connections in UCP

The following sections are included in this chapter:

- [About Borrowing Connections from UCP](#)
- [Setting Connection Pool Properties for UCP](#)
- [Overview of Validating Connections in UCP](#)
- [Returning Borrowed Connections to UCP](#)
- [Removing Connections from UCP](#)
- [UCP Integration with Third-Party Products](#)

3.1 About Borrowing Connections from UCP

An application borrows connections using a pool-enabled data source. This section describes the following concepts about borrowing connections:

- [Overview of Borrowing Connections from UCP](#)
- [Using the Pool-Enabled Data Source](#)
- [Using the Pool-Enabled XA Data Source](#)
- [Setting Connection Properties](#)
- [Using JNDI to Borrow a Connection](#)
- [About Connection Initialization Callback](#)

Note

The instructions in this section use a pool-enabled data source to implicitly create and start a connection pool.

3.1.1 Overview of Borrowing Connections from UCP

The UCP API provides two pool-enabled data sources, one for borrowing regular connections and one for borrowing XA connections. These data sources provide access to UCP JDBC connection pool functionality and include a set of `getConnection` methods that are used to borrow connections. The same pool features are included in both XA and non-XA UCP JDBC connection pools.

UCP JDBC connection pools maintain both available connections and borrowed connections. A connection is reused from the pool if an application requests to borrow a connection that matches an available connection. A new connection is created if no available connection in the pool match the requested connection. The number of available connections and borrowed connections are subjected to pool properties such as pool size, timeout intervals, and validation rules.

3.1.2 Using the Pool-Enabled Data Source

UCP provides a pool-enabled data source (`oracle.ucp.jdbc.PoolDataSource`) that is used to get connections to a database. The `oracle.ucp.jdbc.PoolDataSourceFactory` factory class provides a `getPoolDataSource()` method that creates the pool-enabled data source instance. For example:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
```

The pool-enabled data source requires a connection factory class in order to get an actual physical connection. The connection factory is typically provided as part of a JDBC driver and can be a data source itself. A UCP JDBC connection pool can use any JDBC driver to create physical connections that are then maintained by the pool. The `setConnectionFactoryClassName(String)` method is used to define the connection factory for the pool-enabled data source instance. The following example uses Oracle's `oracle.jdbc.pool.OracleDataSource` connection factory class included with the JDBC driver. If you are using a different vendor's JDBC driver, refer to the vendor's documentation for an appropriate connection factory class.

```
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
```

In addition to the connection factory class, a pool-enabled data source requires the URL, user name, and password that is used to connect to a database. A pool-enabled data source instance includes methods to set each of these properties. The following example uses an Oracle JDBC Thin driver syntax. If you are using a different vendor's JDBC driver, refer to the vendor's documentation for the appropriate URL syntax to use.

```
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");  
pds.setUser("user");  
pds.setPassword("password");
```

See Also

Oracle Database JDBC Developer's Guide for detailed Oracle URL syntax usage.

Lastly, a pool-enabled data source provides a set of `getConnection` methods. The methods include:

- `getConnection()` : Returns a connection that is associated with the user name and password that was used to connect to the database.
- `getConnection(String username, String password)`: Returns a connection that is associated with the given user name and password.
- `getConnection(java.util.Properties labels)`: Returns a connection that matches a given label.
- `getConnection(String username, String password, java.util.Properties labels)` : Returns a connection that is associated with a given user name and password and that matches a given label.

An application uses the `getConnection` methods to borrow a connection handle from the pool that is of the type `java.sql.Connection`. If a connection handle is already in the pool that matches the requested connection (same URL, user name, and password) then it is returned to the application; or else, a new connection is created and a new connection handle is returned to the application. An example for both Oracle and MySQL are provided.

Oracle Example

The following example demonstrates borrowing a connection when using the JDBC Thin driver:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");
pds.setUser("<user>");
pds.setPassword("<password>");

Connection conn = pds.getConnection();
```

MySQL Example

The following example demonstrates borrowing a connection when using the Connector/J JDBC driver from MySQL:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionFactoryClassName("com.mysql.jdbc.jdbc2.optional.
    MysqlDataSource");
pds.setURL("jdbc:mysql://host:3306/dbname");
pds.setUser("<user>");
pds.setPassword("<password>");

Connection conn = pds.getConnection();
```

3.1.3 Using the Pool-Enabled XA Data Source

UCP provides a pool-enabled XA data source (`oracle.ucp.jdbc.PoolXADataSource`) that is used to get XA connections that can be enlisted in a distributed transaction. UCP JDBC XA pools have the same features as non-XA UCP JDBC pools. The `oracle.ucp.jdbc.PoolDataSourceFactory` factory class provides a `getPoolXADataSource()` method that creates the pool-enabled XA data source instance. For example:

```
PoolXADataSource pds = PoolDataSourceFactory.getPoolXADataSource();
```

A pool-enabled XA data source instance, like a non-XA data source instance, requires the connection factory, URL, user name, and password in order to get an actual physical connection. These properties are set in the same way as a non-XA data source instance (see above). However, an XA-specific connection factory class is required to get XA connections. The XA connection factory is typically provided as part of a JDBC driver and can be a data source itself. The following example uses Oracle's `oracle.jdbc.xa.client.OracleXADataSource` XA connection factory class included with the JDBC driver. If a different vendor's JDBC driver is used, refer to the vendor's documentation for an appropriate XA connection factory class.

```
pds.setConnectionFactoryClassName("oracle.jdbc.xa.client.OracleXADataSource");
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");
pds.setUser("user");
pds.setPassword("password");
```

Lastly, a pool-enabled XA data source provides a set of `getXAConnection` methods that are used to borrow a connection handle from the pool that is of the type `javax.sql.XAConnection`. The `getXAConnection` methods are the same as the `getConnection` methods previously described. The following example demonstrates borrowing an XA connection.

```
PoolXADataSource pds = PoolDataSourceFactory.getPoolXADataSource();

pds.setConnectionFactoryClassName("oracle.jdbc.xa.client.OracleXADataSource");
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");
pds.setUser("<user>");
pds.setPassword("<password>");

XAConnection conn = pds.getXAConnection();
```

Related Topics

- [Labeling Connections in UCP](#)

3.1.4 Setting Connection Properties

Oracle's connection factories support properties that configure connections with specific features. UCP pool-enabled data sources provide the `setConnectionProperties(Properties)` method, which is used to set properties on a given connection factory. The following example demonstrates setting connection properties for Oracle's JDBC driver. If you are using a JDBC driver from a different vendor, then refer to the vendor-specific documentation to check whether setting properties in this manner is supported and what properties are available:

```
Properties connProps = new Properties();
connProps.put("fixedString", false);
connProps.put("remarksReporting", false);
connProps.put("restrictGetTables", false);
connProps.put("includeSynonyms", false);
connProps.put("defaultNChar", false);
connProps.put("AccumulateBatchResult", false);

pds.setConnectionProperties(connProps);
```

The UCP JDBC connection pool does not remove connections that are already created if `setConnectionProperties` is called after the pool is created and in use.

① See Also

Oracle Database JDBC Java API Reference for a detailed list of supported properties to configure the connection. For example, to set the auto-commit mode, you can use the `OracleConnection.CONNECTION_PROPERTY_AUTOCOMMIT` property.

3.1.5 Using JNDI to Borrow a Connection

A connection can be borrowed from a connection pool by performing a JNDI look up for a pool-enabled data source and then calling `getConnection()` on the returned object. The pool-enabled data source must first be bound to a JNDI context and a logical name. This assumes that an application includes a Service Provider Interface (SPI) implementation for a naming and directory service where object references can be registered and located.

The following example uses Sun's file system JNDI service provider, which can be downloaded from the JNDI software download page:

<http://www.oracle.com/technetwork/java/index.html>

The example demonstrates creating an initial context and then performing a lookup for a pool-enabled data source that is bound to the name `MyPooledDataSource`. The object returned is then used to borrow a connection from the connection pool.

```
Hashtable env = new Hashtable();
env.put(Context.INITIAL_CONTEXT_FACTORY,
    "com.sun.jndi.fscontext.RefFSContextFactory");
env.put(Context.PROVIDER_URL, "file:/tmp");

ctx = new InitialContext(env);

PoolDataSource jpds = (PoolDataSource)ctx.lookup(MyPooledDataSource);
Connection conn = jpds.getConnection();
```

In the example, `MyPoolDataSource` must be bound to the context. For example:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");
pds.setUser("<user>");
pds.setPassword("<password>");

ctx.bind(MyPooledDataSource, pds);
```

3.1.6 About Connection Initialization Callback

The Connection Initialization Callback enables applications and frameworks to initialize connections retrieved from Universal Connection Pool. It is executed at every connection checkout from the pool, as well as at each successful reconnection during failover.

This section discusses initialization callbacks in the following sections:

- [Overview of Connection Initialization Callback](#)
- [Creating an Initialization Callback](#)
- [Registering an Initialization Callback](#)
- [Removing or Unregistering an Initialization Callback](#)

3.1.6.1 Overview of Connection Initialization Callback

If an application cannot use connection labeling because it cannot be changed, then the connection initialization callback is provided for such an application.

When registered, the initialization callback is executed every time a connection is borrowed from the pool and at each successful reconnection following a recoverable error. Using the same callback at both run time and replay ensures that exactly the same initialization, which was used when the original session was established, is reestablished at run time. If the callback invocation fails, then replay is disabled on that connection.

3.1.6.2 Creating an Initialization Callback

To create a UCP connection initialization callback, an application implements the `oracle.ucp.jdbc.ConnectionInitializationCallback` interface. This interface has the following method:

```
void initialize(java.sql.Connection connection) throws SQLException;
```

Note

- One callback is created for every connection pool.
- This callback is not used if a labeling callback is registered for the connection pool.

Example

The following example demonstrates how to create a simple initialization callback:

```
import oracle.ucp.jdbc.ConnectionInitializationCallback;
class MyConnectionInitializationCallback implements ConnectionInitializationCallback
{
    public MyConnectionInitializationCallback()
    {
        ...
    }
    public void initialize(java.sql.Connection connection) throws SQLException
    {
        // Reset the state for the connection, if necessary (like ALTER SESSION)
    }
}
```

3.1.6.3 Registering an Initialization Callback

UCP provides the `registerConnectionInitializationCallback` method in the `oracle.ucp.jdbc.PoolDataSource` interface for registering a connection initialization callback.

```
public void registerConnectionInitializationCallback (ConnectionInitializationCallback cbk) throws SQLException;
```

One callback may be registered on each connection pool instance.

3.1.6.4 Removing or Unregistering an Initialization Callback

UCP provides the `unregisterConnectionInitializationCallback` method in the `oracle.ucp.jdbc.PoolDataSource` interface for unregistering a connection initialization callback.

```
public void unregisterConnectionInitializationCallback (ConnectionInitializationCallback cbk) throws SQLException;
```

See Also

Oracle Universal Connection Pool Java API Reference for more information

3.2 Setting Connection Pool Properties for UCP

UCP JDBC connection pools are configured using connection pool properties. The properties have `get` and `set` methods that are available through a pool-enabled data source instance. The methods are a convenient way to programmatically configure a pool. If no pool properties are set, then a connection pool uses default property values.

The following example demonstrates configuring connection pool properties. The example sets the connection pool name and the maximum/minimum number of connections allowed in the pool.

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionPoolName("JDBC_UCP");
pds.setMinPoolSize(4);pds.setMaxPoolSize(20);
```

UCP JDBC connection pool properties may be set in any order and can be dynamically changed at run time. For example, `setMaxPoolSize` could be changed at any time and the pool recognizes the new value and adapts accordingly.

Related Topics

- [Optimizing Universal Connection Pool Behavior](#)

3.3 Overview of Validating Connections in UCP

Connections can be validated using pool properties when the connection is borrowed, and also programmatically using the `ValidConnection` interface. Both approaches are detailed in this section. Invalid connections can affect application performance and availability.

3.3.1 Validating When Borrowing

A connection can be validated by executing a SQL statement on a connection when the connection is borrowed from the connection pool. Two connection pool properties are used in conjunction in order to enable connection validation:

- `setValidateConnectionOnBorrow(boolean)`: Specifies whether or not connections are validated when the connection is borrowed from the connection pool. The method enables validation for every connection that is borrowed from the pool. A value of `false` means no validation is performed. The default value is `false`.
- `setSQLForValidateConnection(String)`: Specifies the SQL statement that is executed on a connection when it is borrowed from the pool.

Note

The `setSQLForValidateConnection` property is not recommended when using an Oracle JDBC driver. UCP performs an internal ping when using an Oracle JDBC driver. The mechanism is faster than executing an SQL statement and is overridden if this property is set. Instead, set the `setValidateConnectionOnBorrow` property to `true` and do not include the `setSQLForValidateConnection` property.

The following example demonstrates validating a connection when borrowing the connection from the pool. The example uses Connector/J JDBC driver from MySQL:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionFactoryClassName("com.mysql.jdbc.jdbc2.optional.
    MysqlDataSource");
pds.setURL("jdbc:mysql://host:3306/mysql");
pds.setUser("<user>");
pds.setPassword("<password>");

pds.setValidateConnectionOnBorrow(true);
pds.setSQLForValidateConnection("select * from mysql.user");

Connection conn = pds.getConnection();
```

See Also[Minimizing Connection Request Delay](#)

3.3.2 Minimizing Connection Validation with `setSecondsToTrustIdleConnection()` Method

In UCP, when you set the value of the `setValidateConnectionOnBorrow(boolean)` method to `true`, then each connection is validated during the checkout. This validation may incur significant overhead in applications that checkout database connections frequently.

To minimize the impact of frequent connection validation, you can now set the `setSecondsToTrustIdleConnection(int)` method with an appropriate value to trust recently-used or recently-tested database connections. Setting this value skips the connection validation test and improves application performance significantly.

The following table describes the new methods available in Oracle Database 19c Release for using this feature:

Method	Description
<code>setSecondsToTrustIdleConnection(int secondsToTrustIdleConnection)</code>	Sets the time in seconds to trust a recently-used or recently-tested database connection and skip the validation test during connection checkout.
<code>getSecondsToTrustIdleConnection()</code>	Retrieves the value that was set using the <code>setSecondsToTrustIdleConnection(int)</code> method.

When you set the `setSecondsToTrustIdleConnection(int)` method to a positive value, then the connection validation is skipped, if the connection was used within the time specified in the `secondsToTrustIdleConnection(int)` method. The default value is 0 seconds, which means that the feature is disabled.

Note

The `setSecondsToTrustIdleConnection(int)` method works only if the `setValidateConnectionOnBorrow(boolean)` method is set to `true`. If you set the `setSecondsToTrustIdleConnection(int)` method to a non-zero value, without setting the `setValidateConnectionOnBorrow(boolean)` method to `true`, then UCP throws the following exception:

Invalid seconds to trust idle connection value or usage.

3.3.3 Checking If a Connection Is Valid

The `oracle.ucp.jdbc.ValidConnection` interface provides two methods: `isValid` and `setInvalid`. The `isValid` method returns whether or not a connection is usable and the `setInvalid` method is used to indicate that a connection should be removed from the pool instance.

The `isValid` method is used to check if a connection is still usable after an SQL exception has been thrown. This method can be used at any time to check if a borrowed connection is valid. The method is particularly useful in combination with a retry mechanism, such as the Fast Connection Failover actions that are triggered after a down event of Oracle RAC.

Note

- The `isValid` method checks with the pool instance and Oracle JDBC driver to determine whether a connection is still valid. The `isValid` method results in a round-trip to the database only if both the pool and the driver report that a connection is still valid. The round-trip is used to check for database failures that are not immediately discovered by the pool or the driver.
- Starting from Oracle Database Release 18c, there is a new variant of the `isValid` method that sends an empty packet to the database unlike the older version of the method that uses a ping-pong protocol and makes a full round-trip to the database.

See Also

Oracle Database JDBC Developer's Guide

The `isValid` method is also helpful when used in conjunction with the connection timeout and connection harvesting features. These features may return a connection to the pool while a connection is still held by an application. In such cases, the `isValid` method returns `false`, allowing the application to get a new connection.

The following example demonstrates using the `isValid` method:

```
try { conn = poolDataSource.getConnection ... } catch (SQLException sqlExc)
{
    if (conn == null || !((ValidConnection) conn).isValid())

        // take the appropriate action

    ...
    conn.close();
}
```

For XA applications, before calling the `isValid()` method, you must cast any `XAConnection` that is obtained from `PoolXADataSource` to a `ValidConnection`. If you cast a `Connection` that is obtained by calling the `XAConnection.getConnection()` method to `ValidConnection`, then it may throw an exception.

Related Topics

- [Using Oracle RAC Features](#)

Related Topics

- [Removing Connections from UCP](#)

3.4 Returning Borrowed Connections to UCP

Borrowed connections that are no longer being used should be returned to the pool so that they can be available for the next connection request. The `close` method closes connections and

automatically returns them to the pool. The `close` method does not physically remove the connection from the pool.

Borrowed connections that are not closed will remain borrowed; subsequent requests for a connection result in a new connection being created if no connections are available. This behavior can cause many connections to be created and can affect system performance.

The following example demonstrates closing a connection and returning it to the pool:

```
Connection conn = pds.getConnection();

//do some work with the connection.

conn.close();
conn=null;
```

3.5 Removing Connections from UCP

The `setInvalid` method of the `ValidConnection` interface indicates that a connection should be removed from the connection pool when it is closed. The method is typically used when a connection is no longer usable, such as after an exception or if the `isValid` method of the `ValidConnection` interface returns false. The method can also be used if an application deems the state on a connection to be bad. The following example demonstrates using the `setInvalid` method to close and remove a connection from the pool:

```
Connection conn = pds.getConnection();
...

((ValidConnection) conn).setInvalid();
...

conn.close();
conn=null;
```

3.6 UCP Integration with Third-Party Products

Third-party products, such as middleware platforms or frameworks, can use UCP to provide connection pooling functionality for their applications and services. UCP integration includes the same connection pool features that are available to stand-alone applications and offers the same tight integration with the Oracle Database.

Two data source classes are available as integration points with UCP: `PoolDataSourceImpl` for non-XA connection pools and `PoolXADataSourceImpl` for XA connection pools. Both classes are located in the `oracle.ucp.jdbc` package. These classes are implementations of the `PoolDataSource` and `PoolXADataSource` interfaces, respectively, and contain default constructors.

See Also

Oracle Universal Connection Pool Java API Reference for more information on the implementation classes.

These implementations explicitly create connection pool instances and can return connections. For example:

```
PoolXADataSource pds = new PoolXADataSourceImpl();

pds.setConnectionFactoryClassName("oracle.jdbc.xa.client.OracleXADataSource");
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");
pds.setUser("user");
pds.setPassword("password");

XAConnection conn = pds.getXAConnection();
```

Third-party products can instantiate these data source implementation classes. In addition, the methods of these interfaces follow the JavaBean design pattern and can be used to set connection pool properties on the class using reflection. For example, a UCP data source that uses an Oracle JDBC connection factory and database might be defined as follows and loaded into a JNDI registry:

```
<data-sources>
<data-source
  name="UCPDataSource"
  jndi-name="jdbc/UCP_DS"
  data-source-class="oracle.ucp.jdbc.PoolDataSourceImpl">
  <property name="ConnectionFactoryClassName"
    value="oracle.jdbc.pool.OracleDataSource"/>
  <property name="URL" value="jdbc:oracle:thin:@//localhost:1521:oracle"/>
  <property name="User" value="user"/>
  <property name="Password" value="password"/>
  <property name="ConnectionPoolName" value="MyPool"/>
  <property name="MinPoolSize" value="5"/>
  <property name="MaxPoolSize" value="50"/>
</data-source>
</data-sources>
```

When using reflection, the name attribute matches (case sensitive) the name of the setter method used to set the property. An application could then use the data source as follows:

```
Connection connection = null;
try {
  InitialContext context = new InitialContext();
  DataSource ds = (DataSource) context.lookup("jdbc/UCP_DS");
  connection = ds.getConnection();
  ...
}
```

4

Optimizing Universal Connection Pool Behavior

This chapter describes the following concepts:

- [Optimizing Connection Pools](#)
- [About Controlling the Pool Size in UCP](#)
- [About Optimizing Real-World Performance with Static Connection Pools](#)
- [Stale Connections in UCP](#)
- [About Harvesting Connections in UCP](#)
- [About Caching SQL Statements in UCP](#)

4.1 Optimizing Connection Pools

This section provides instructions for setting connection pool properties in order to optimize pooling behavior. Upon creation, UCP JDBC connection pools are pre-configured with a default setup. The default setup provides a general, all-purpose connection pool. However, different applications may have different database connection requirements and may want to modify the default behavior of the connection pool. Behaviors, such as pool size and connection timeouts can be configured and can improve overall connection pool performance as well as connection availability. In many cases, the best way to tune a connection pool for a specific application is to try different property combinations using different values until optimal performance and throughput is achieved.

Setting Connection Pool Properties

Connection pool properties are set either when getting a connection through a pool-enabled data source or when creating a connection pool using the connection pool manager.

The following example demonstrates setting connection pool properties through a pool-enabled data source:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionPoolName("JDBC_UCP");
pds.setMinPoolSize(4);pds.setMaxPoolSize(20);
...
```

The following example demonstrates setting connection pool properties when creating a connection pool using the connection pool manager:

```
UniversalConnectionPoolManager mgr = UniversalConnectionPoolManagerImpl.
getUniversalConnectionPoolManager();

pds.setConnectionPoolName("JDBC_UCP");
pds.setMinPoolSize(4);pds.setMaxPoolSize(20);
...

mgr.createConnectionPool(pds);
```

4.2 About Controlling the Pool Size in UCP

UCP JDBC connection pools include a set of properties that are used to control the size of the pool. The properties allow the number of connections in the pool to increase and decrease as demand increases and decreases. This dynamic behavior helps conserve system resources that are otherwise lost on maintaining unnecessary connections.

This section describes the following topics:

- [Setting the Initial Pool Size](#)
- [Setting the Minimum Pool Size](#)
- [Setting the Maximum Pool Size](#)

4.2.1 Setting the Initial Pool Size

The initial pool size property specifies the number of available connections that are created when the connection pool is initially created or re-initialized. This property is typically used to reduce the ramp-up time incurred by priming the pool to its optimal size.

A value of 0 indicates that no connections are pre-created. The default value is 0. The following example demonstrates configuring an initial pool size:

```
pds.setInitialPoolSize(5);
```

If the initial pool size property is greater than the maximum pool size property, then only the maximum number of connections are initialized.

If the initial pool size property is less than the minimum pool size property, then only the initial number of connections are initialized and maintained until enough connections are created to meet the minimum pool size value.

4.2.2 Setting the Minimum Pool Size

The minimum pool size property specifies the minimum amount of available connections and borrowed connections that a pool maintains. A connection pool always tries to return to the minimum pool size specified unless the minimum amount is yet to be reached. For example, if the minimum limit is set to 10 and only 2 connections are ever created and borrowed, then the number of connections maintained by the pool remains at 2 because this number is less than the minimum pool size.

This property allows the number of connections in the pool to decrease as demand decreases. At the same time, the property ensures that system resources are not wasted on maintaining connections that are unnecessary.

The default value is 0. The following example demonstrates configuring a minimum pool size:

```
pds.setMinPoolSize(2);
```

4.2.3 Setting the Maximum Pool Size

The maximum pool size property specifies the maximum number of available and borrowed (in use) connections that a pool maintains. If the maximum number of connections are borrowed, no connections will be available until a connection is returned to the pool.

This property allows the number of connections in the pool to increase as demand increases. At the same time, the property ensures that the pool does not grow to the point of exhausting the resources of a system, which ultimately affects the performance and availability of an application.

A value of 0 indicates that no connections are maintained by the pool. An attempt to get a connection results in an exception. The default value is to allow the pool to continue to create connections up to `Integer.MAX_VALUE` (2147483647 by default). The following example demonstrates configuring a maximum pool size:

```
pds.setMaxPoolSize(100);
```

4.3 Real-World Performance Considerations with Respect to Connection Pool Sizes

Most on-line transaction processing (OLTP) performance problems that the Real-World Performance group investigates relate to the connection strategy used by the application. For this reason, designing a sound connection strategy is crucial for system performance, especially in enterprise environments that must scale to meet increasing demand.

Most applications use a dynamic pool of connections to the database, configured with a minimum number of connections to keep open on the database and a maximum number of connections that can be made to the database. When an application needs a connection to the database, then it requests one from the pool. If there are no connections available, then the application creates a new connection, if it has not reached the maximum number of connections already. If a connection has not been used for a specified duration of time, then the application closes the connection, if there are more than the minimum number of connections available.

This configuration conserves system resources by only maintaining the number of connections actively needed by the application. In the real world, this configuration enables connection storms and database system CPU over-subscription, quickly destabilizing a system. A connection storm can occur when there are lots of activities on the application server requiring database connections. If there are not enough connections to the database to serve all of the requests, then the application server opens new connections. Creating a new connection to the database is a resource intensive activity, and when lots of connections are made in a short period of time, it can overwhelm the CPU resources on the database system.

So, for creating connection pools, the total number of connections to the database from all pools must be based on the CPU cores available on the system. Oracle recommends maximum five connections per CPU core. The ideal number varies depending on the application and the system hardware. However, the value is somewhere within that range. The Real-World Performance group recommends creating pools that:

- Have a minimum size that is close to the actual minimum number of connections in use
- Have a maximum size that is not too high and follows the guideline of having five connections per CPU core
- Do not have too much difference between the maximum value and the minimum value, so that connection storms can be avoided

For example, if a database server has 2 CPUs and 12 cores per CPU, then there are 24 cores available and the number of connections to the database, at any given point of time, should be less than 120. This number is cumulative for all applications and all databases connecting to the system, if there are more than one databases on the system. If there are two application servers, then the maximum number of connections (for example, 120 in this case) should be

divided between them. If there are two databases running on the system, then the maximum number of connections should be divided between them.

① See Also

- <https://www.youtube.com/watch?v=Oo-tBpVewP4>
- <https://www.youtube.com/watch?v=XzN8Rp6gIEo>

4.4 Stale Connections in UCP

Stale connections are connections that remain either available or borrowed, but are no longer being used. Stale connections that remain borrowed may affect connection availability. In addition, stale connections may impact system resources that are otherwise wasted on maintaining unused connections for extended periods of time. The pool properties discussed in this section are used to control stale connections.

This section describes the following topics:

- [What is Connection Reuse?](#)
- [Setting the Abandon Connection Timeout](#)
- [Setting the Time-To-Live Connection Timeout](#)
- [Setting the Connection Wait Timeout](#)
- [Setting the Inactive Connection Timeout](#)
- [Setting the Query Timeout](#)
- [Setting the Timeout Check Interval](#)

① Note

It is good practice to close all connections that are no longer required by an application. Closing connections helps minimize the number of stale connections that remain borrowed.

4.4.1 What is Connection Reuse?

The connection reuse feature allows connections to be gracefully closed and removed from a connection pool after a specific amount of time or after the connection has been used a specific number of times. This feature saves system resources that are otherwise wasted on maintaining unusable connections.

4.4.1.1 Setting the Maximum Connection Reuse Time

The maximum connection reuse time allows connections to be gracefully closed and removed from the pool after a connection has been in use for a specific amount of time. The timer for this property starts when a connection is physically created. Borrowed connections are closed only after they are returned to the pool and the reuse time is exceeded.

This feature is typically used when a firewall exists between the pool tier and the database tier and is setup to block connections based on time restrictions. The blocked connections remain

in the pool even though they are unusable. In such scenarios, the connection reuse time is set to a smaller value than the firewall timeout policy.

① Note

The maximum connection reuse time is different from the time-to-live connection timeout. The time-to-live connection timeout starts when a connection is borrowed from the pool; while, the maximum connection reuse time starts when the connection is physically created. In addition, with a time-to-live timeout, a connection is closed and returned to the pool for reuse if the timeout expires during the borrowed period. With maximum connection reuse time, a connection is closed and discarded from the pool after the timeout expires.

The maximum connection reuse time value represents seconds. A value of 0 indicates that this feature is disabled. The default value is 0. The following example demonstrates configuring a maximum connection reuse time:

```
pds.setMaxConnectionReuseTime(300);
```

Related Topics

- [Setting the Time-To-Live Connection Timeout](#)

4.4.1.2 Setting the Maximum Connection Reuse Count

The maximum connection reuse count allows connections to be gracefully closed and removed from the connection pool after a connection has been borrowed a specific number of times. This property is typically used to periodically recycle connections in order to eliminate issues such as memory leaks.

A value of 0 indicates that this feature is disabled. The default value is 0. The following example demonstrates configuring a maximum connection reuse count:

```
pds.setMaxConnectionReuseCount(100);
```

4.4.2 Setting the Connection Validation Timeout

The connection validation timeout specifies the duration within which a borrowed connection from the pool is validated. This is the maximum time for a connection validation operation. If the validation is not completed during this period, then the connection is treated as invalid.

The connection validation timeout value represents seconds. The default value is set to 15. The following example demonstrates configuring a connection validation timeout:

```
pd.setConnectionValidationTimeout(55);
```

4.4.3 Setting the Abandon Connection Timeout

The abandoned connection timeout (ACT) enables borrowed connections to be reclaimed back into the connection pool after a connection has not been used for a specific amount of time. Abandonment is determined by monitoring calls to the database. This timeout feature helps maximize connection reuse and conserves system resources that are otherwise lost on maintaining borrowed connections that are no longer in use.

Note

UCP either cancels or rolls back connections that have local transactions pending before reclaiming connections for reuse.

The ACT value represents seconds. A value of 0 indicates that the feature is disabled. The default value is set to 0. The following example demonstrates configuring an abandoned connection timeout:

```
pds.setAbandonedConnectionTimeout(10);
```

Every connection is reaped after a specific period of time. Either it is reaped when ACT expires, or, if it is immune from ACT, then it is reaped after the immunity expires. If you set ACT on a pool, then:

- If a statement is executed without calling the `Statement.setQueryTimeout` method on that statement, then the connection is reaped if ACT is exceeded, even though the connection is waiting for the server to respond to the query.
- If a statement is executed with calling the `Statement.setQueryTimeout` method, then the connection is reaped after the query timeout and ACT have expired. The connection is not reaped while waiting on the query timeout. The expiration of the query timeout is an event that resets the ACT timer. If the ACT expires while waiting for the cancel action that occurs at the expiration of the query time out, then the connection is reaped.
- If a connection has two statements: s1 with a query timeout and s2 without a query timeout, then ACT does not reap the connection while s1 waits for the query timeout, but reaps the connection if s2 hangs.

Note that the two statements execute sequentially based on JDBC requirement.

4.4.4 Setting the Time-To-Live Connection Timeout

The time-to-live connection timeout enables borrowed connections to remain borrowed for a specific amount of time before the connection is reclaimed by the pool. This timeout feature helps maximize connection reuse and helps conserve systems resources that are otherwise lost on maintaining connections longer than their expected usage.

Note

UCP either cancels or rolls back connections that have local transactions pending before reclaiming connections for reuse.

The time-to-live connection timeout value represents seconds. A value of 0 indicates that the feature is disabled. The default value is set to 0. The following example demonstrates configuring a time-to-live connection timeout:

```
pds.setTimeToLiveConnectionTimeout(18000)
```

4.4.5 Setting the Connection Wait Timeout

The connection wait timeout specifies how long an application request waits to obtain a connection if there are no longer any connections in the pool. A connection pool runs out of

connections if all connections in the pool are being used (borrowed) and if the pool size has reached its maximum connection capacity as specified by the maximum pool size property. The request receives an SQL exception if the timeout value is reached. The application can then retry getting a connection. This timeout feature improves overall application usability by minimizing the amount of time an application is blocked and provides the ability to implement a graceful recovery.

The connection wait timeout value represents seconds. A value of 0 indicates that the feature is disabled. The default value is set to 3 seconds. The following example demonstrates configuring a connection wait timeout:

```
pds.setConnectionWaitTimeout(10);
```

4.4.6 Setting the Inactive Connection Timeout

The inactive connection timeout specifies how long an available connection can remain idle before it is closed and removed from the pool. This timeout property is only applicable to available connections and does not affect borrowed connections. This property helps conserve resources that are otherwise lost on maintaining connections that are no longer being used. The inactive connection timeout (together with the maximum pool size) allows a connection pool to grow and shrink as application load changes.

The inactive connection timeout value represents seconds. A value of 0 indicates that the feature is disabled. The default value is set to 0. The following example demonstrates configuring an inactive connection timeout:

```
pds.setInactiveConnectionTimeout(60);
```

4.4.7 Setting the Query Timeout

In Oracle Database 12c Release 2 (12.2.0.1), UCP introduced the `queryTimeout` property. This property specifies the number of seconds UCP waits for a `Statement` object to execute. If the limit is exceeded, then a `DatabaseException` is thrown. Use the `setQueryTimeout` method for setting this property in the following way:

```
...
PoolDataSourceImpl pds = new PoolDataSourceImpl();
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
pds.setURL(<url>);
pds.setUser("scott");
pds.setPassword(<password>);
pds.setConnectionPoolName("my_pool");
pds.setQueryTimeout(60); // 60 seconds to wait on query
...
```

4.4.8 Setting the Timeout Check Interval

The timeout check interval property controls how frequently the timeout properties (abandoned connection timeout, time-to-live connection timeout, and inactive connection timeout) are enforced. Connections that have timed-out are reclaimed when the timeout check cycle runs. This means that a connection may not actually be reclaimed by the pool at the moment that the connection times-out. The lag time between the connection timeout and actually reclaiming the connection may be considerable depending on the size of the timeout check interval.

The timeout check interval property represents seconds. The default value is set to 30. The following example demonstrates configuring a property check interval:

```
pds.setTimeoutCheckInterval(60);
```

See Also

Oracle Database Net Services Administrator's Guide for more information about Oracle Net Services

4.5 About Harvesting Connections in UCP

The connection harvesting feature allows a specified number of borrowed connections to be reclaimed when the connection pool reaches a specified number of available connections. This section describes the following concepts:

- [Overview of Harvesting Connections in UCP](#)
- [Setting a Connection to Harvestable](#)
- [Setting the Harvest Trigger Count](#)
- [Setting the Harvest Maximum Count](#)

4.5.1 Overview of Harvesting Connections in UCP

This feature helps ensure that a certain number of connections are always available in the pool and helps maximize performance. The feature is particularly useful if an application caches connection handles. Caching is typically performed for performance reasons because it minimizes re-initialization of state necessary for connections to participate in a transaction.

For example, a connection is borrowed from the pool, initialized with necessary session state, and then held in a context object. Holding connections in this manner may cause the connection pool to run out of available connections. The connection harvest feature reclaims the borrowed connections, if appropriate, and allows the connections to be reused.

Connection harvesting is controlled using the `HarvestableConnection` interface and configured or enabled using two pool properties: Connection Harvest Trigger Count and Connection Harvest Maximum Count. The interface and properties are used together when implementing the connection harvest feature.

4.5.2 Setting a Connection to Harvestable

The `setConnectionHarvestable(boolean)` method of the `oracle.ucp.jdbc.HarvestableConnection` interface controls whether or not a connection will be harvested. This method is used as a locking mechanism when connection harvesting is enabled. For example, the method is set to `false` on a connection when the connection is being used within a transaction and must not be harvested. After the transaction completes, the method is set to `true` on the connection and the connection can be harvested if required.

Note

All connections are harvestable, by default, when the connection harvest feature is enabled. If the feature is enabled, the `setConnectionHarvestable` method should always be used to explicitly control whether a connection is harvestable.

The following example demonstrates using the `setConnectionHarvestable` method to indicate that a connection is not harvestable when the connection harvest feature attempts to harvest connections:

```
Connection conn = pds.getConnection();  
  
((HarvestableConnection) conn).setConnectionHarvestable(false);
```

4.5.3 Setting the Harvest Trigger Count

The connection harvest trigger count specifies the available connection threshold that triggers connection harvesting. For example, if the connection harvest trigger count is set to 10, then connection harvesting is triggered when the number of available connections in the pool drops to 10.

A value of `Integer.MAX_VALUE` (2147483647 by default) indicates that connection harvesting is disabled. The default value is `Integer.MAX_VALUE`.

The following example demonstrates enabling connection harvesting by configuring a connection harvest trigger count.

```
pds.setConnectionHarvestTriggerCount(2);
```

4.5.4 Setting the Harvest Maximum Count

The connection harvest maximum count property specifies how many borrowed connections should be returned to the pool once the harvest trigger count has been reached. The number of connections actually harvested may be anywhere from 0 to the connection harvest maximum count value. Least recently used connections are harvested first which allows very active user sessions to keep their connections the most.

The harvest maximum count value can range from 0 to the maximum connection property value. The default value is 1. An `SQLException` is thrown if an out-of-range value is specified.

The following example demonstrates configuring a connection harvest maximum count.

```
pds.setConnectionHarvestMaxCount(5);
```

Note

- If connection harvesting and abandoned connection timeout features are enabled at the same time, then the timeout processing does not reclaim the connections that are designated as nonharvestable.
- If connection harvesting and time-to-live connection timeout features are enabled at the same time, then the timeout processing reclaims the connections that are designated as nonharvestable.

Related Topics

- [Controlling Reclaimable Connection Behavior](#)

4.6 About Caching SQL Statements in UCP

This section describes how to cache SQL statements in UCP, in the following sections:

- [Overview of Statement Caching in UCP](#)
- [Enabling Statement Caching in UCP](#)

4.6.1 Overview of Statement Caching in UCP

Statement caching makes working with statements more efficient. Statement caching improves performance by caching executable statements that are used repeatedly and makes it unnecessary for programmers to explicitly reuse prepared statements. Statement caching eliminates overhead due to repeated cursor creation, repeated statement parsing and creation and reduces overhead of communication between applications and the database. Statement caching and reuse is transparent to an application. Each statement cache is associated with a physical connection. That is, each physical connection will have its own statement cache.

The match criteria for cached statements are as follows:

- The SQL string in the statement must be the same (case-sensitive) to one in the cache.
- The statement type must be the same (prepared or callable) to the one in the cache.
- The scrollable type of result sets produced by the statement must be the same (forward-only or scrollable) as the one in the cache.

Statement caching is implemented and enabled differently depending on the JDBC driver vendor. The instructions in this section are specific to Oracle's JDBC driver. Statement caching on other vendors' drivers can be configured by setting a connection property on a connection factory. Refer to the JDBC vendor's documentation to determine whether statement caching is supported and if it can be set as a connection property. UCP does support JDBC 4.0 (JDK16) APIs to enable statement pooling if a JDBC vendor supports it.

Related Topics

- [Setting Connection Properties](#)

4.6.2 Enabling Statement Caching in UCP

The maximum number of statements property specifies the number of statements to cache for each connection. The property only applies to the Oracle JDBC driver. If the property is not set, or if it is set to 0, then statement caching is disabled. By default, statement caching is disabled. When statement caching is enabled, a statement cache is associated with each physical connection maintained by the connection pool. A single statement cache is not shared across all physical connections.

The following example demonstrates enabling statement caching:

```
pds.setMaxStatements(10);
```

Determining the Statement Cache Size

The cache size should be set to the number of distinct statements the application issues to the database. If the number of statements that an application issues to the database is unknown, use the JDBC performance metrics to assist with determining the statement cache size.

Statement Cache Size Resource Issues

Each connection is associated with its own statement cache. Statements held in a connection's statement cache may hold on to database resources. It is possible that the number of opened connections combined with the number of cached statements for each connection could exceed the limit of open cursors allowed for the database. This issue may be avoided by reducing the number of statements allowed in the cache, or by increasing the limit of open cursors allowed by the database.

5

Labeling Connections in UCP

This chapter discusses the following topics:

- [Overview of Labeling Connections in UCP](#)
- [Implementation of a Labeling Callback in UCP](#)
- [Applying Connection Labels in UCP](#)
- [Borrowing Labeled Connections from UCP](#)
- [Checking Unmatched Labels in UCP](#)
- [Integration of UCP with DRCP](#)
- [Removing a Connection Label in UCP](#)

5.1 Overview of Labeling Connections in UCP

Applications often initialize connections retrieved from a connection pool before using the connection. The initialization varies and could include simple state re-initialization that requires method calls within the application code or database operations that require round trips over the network. The cost of such initialization may be significant.

Labeling connections enables an application to attach arbitrary name/value pairs to a connection. The application can request a connection with the desired label from the connection pool. By associating particular labels with particular connection states, an application can retrieve an already initialized connection from the pool and avoid the time and cost of re-initialization. The connection labeling feature does not impose any meaning on user-defined keys or values; the meaning of user-defined keys and values is defined solely by the application.

Some of the examples for connection labeling include, role, NLS language settings, transaction isolation levels, stored procedure calls, or any other state initialization that is expensive and necessary on the connection before work can be executed by the resource.

Connection labeling is application-driven and requires the use of two interfaces. The `oracle.ucp.jdbc.LabelableConnection` interface is used to apply and remove connection labels, as well as retrieve labels that have been set on a connection. The `oracle.ucp.ConnectionLabelingCallback` interface is used to create a labeling callback that determines whether or not a connection with a requested label already exists. If no connections exist, the interface allows current connections to be configured as required. The methods of these interfaces are described in detail throughout this chapter.

5.2 Implementation of a Labeling Callback in UCP

UCP uses Database Resident Connection Pooling (DRCP) tagging infrastructure to support labeling in UCP, whether you work with single labels or multiple labels. However, the behavior with multiple labels can be a little different when you use the UCP and DRCP combination instead of only UCP.

This section discusses the following topics:

- [When to Use a Labeling Callback in UCP](#)
- [Creating a Labeling Callback in UCP](#)
- [Registering a Labeling Callback in UCP](#)
- [Removing a Labeling Callback in UCP](#)

See Also

["Integration of UCP with DRCP"](#)

5.2.1 When to Use a Labeling Callback in UCP

A labeling callback is used to define how the connection pool selects labeled connections and allows the selected connection to be configured before returning it to an application. Applications that use the connection labeling feature must provide a callback implementation.

A labeling callback is used when a labeled connection is requested but there are no connections in the pool that match the requested labels. The callback determines which connection requires the least amount of work in order to be re-configured to match the requested label and then enables the connection labels to be updated before returning the connection to the application. This section describes the following topics:

5.2.2 Creating a Labeling Callback in UCP

To create a labeling callback, an application implements the `oracle.ucp.ConnectionLabelingCallback` interface. One callback is created per connection pool. The interface provides the following two methods:

- [The cost Method](#)
- [The configure Method](#)

The cost Method

This method projects the cost of configuring connections considering label-matching differences. Upon a connection request, the connection pool uses this method to select a connection with the least configuration cost.

```
public int cost(Properties requestedLabels, Properties currentLabels);
```

The configure Method

This method is called by the connection pool on the selected connection before returning it to the application. The method is used to set the state of the connection and apply or remove any labels to/from the connection.

```
public boolean configure(Properties requestedLabels, Connection conn);
```

The connection pool iterates over each connection available in the pool. For each connection, it calls the `cost` method. The result of the `cost` method is an integer which represents an estimate of the cost required to reconfigure the connection to the required state. The larger the value, the costlier it is to reconfigure the connection. The connection pool always returns connections with the lowest cost value. The algorithm is as follows:

- If the `cost` method returns 0 for a connection, then the connection is a match. The connection pool does not call the `configure` method on the connection found and returns the connection as it is.
- If the `cost` method returns a value greater than 0, then the connection pool iterates until it finds a connection with a cost value of 0 or runs out of available connections.
- If the pool has iterated through all available connections and the lowest cost of a connection is `Integer.MAX_VALUE` (2147483647 by default), then no connection in the pool is able to satisfy the connection request. The pool creates and returns a new connection. If the pool has reached the maximum pool size (it cannot create a new connection), then the pool either throws an SQL exception or waits if the connection wait timeout attribute is specified.
- If the pool has iterated through all available connections and the lowest cost of a connection is less than `Integer.MAX_VALUE`, then the `configure` method is called on the connection and the connection is returned. If multiple connections are less than `Integer.MAX_VALUE`, the connection with the lowest cost is returned.

Note

A cost of 0 does not imply that `requestedLabels` equals `currentLabels`.

5.2.2.1 Example of Labeling Callback in UCP

The following example demonstrates a simple labeling callback implementation that implements both the `cost` and `configure` methods. The callback is used to find a labeled connection that is initialized with a specific transaction isolation level.

```
class MyConnectionLabelingCallback
    implements ConnectionLabelingCallback {

    public MyConnectionLabelingCallback()
    {
    }

    public int cost(Properties reqLabels, Properties currentLabels)
    {
        // Case 1: exact match
        if (reqLabels.equals(currentLabels))
        {
            System.out.println("## Exact match found!! ##");
            return 0;
        }

        // Case 2: some labels match with no unmatched labels
        String iso1 = (String) reqLabels.get("TRANSACTION_ISOLATION");
        String iso2 = (String) currentLabels.get("TRANSACTION_ISOLATION");
        boolean match =
            (iso1 != null && iso2 != null && iso1.equalsIgnoreCase(iso2));
        Set rKeys = reqLabels.keySet();
        Set cKeys = currentLabels.keySet();
        if (match && rKeys.containsAll(cKeys))
        {
            System.out.println("## Partial match found!! ##");
            return 10;
        }
    }
}
```

```

// No label matches to application's preference.
// Do not choose this connection.
System.out.println("## No match found!! ##");
return Integer.MAX_VALUE;
}

public boolean configure(Properties reqLabels, Object conn)
{
    try
    {
        String isoStr = (String) reqLabels.get("TRANSACTION_ISOLATION");
        ((Connection)conn).setTransactionIsolation(Integer.valueOf(isoStr));
        LabelableConnection lconn = (LabelableConnection) conn;

        // Find the unmatched labels on this connection
        Properties unmatchedLabels =
            lconn.getUnmatchedConnectionLabels(reqLabels);

        // Apply each label <key,value> in unmatchedLabels to conn
        for (Map.Entry<Object, Object> label : unmatchedLabels.entrySet())
        {
            String key = (String) label.getKey();
            String value = (String) label.getValue();
            lconn.applyConnectionLabel(key, value);
        }
    }
    catch (Exception exc)
    {
        return false;
    }
    return true;
}
}

```

5.2.3 Registering a Labeling Callback in UCP

A pool-enabled data source provides the `registerConnectionLabelingCallback(ConnectionLabelingCallback callback)` method for registering labeling callbacks. Only one callback may be registered on a connection pool. The following example demonstrates registering a labeling callback that is implemented in the `MyConnectionLabelingCallback` class:

```

MyConnectionLabelingCallback callback = new MyConnectionLabelingCallback();
pds.registerConnectionLabelingCallback( callback );

```

5.2.4 Removing a Labeling Callback in UCP

A pool-enabled data source provides the `removeConnectionLabelingCallback()` method for removing a labeling callback. The following example demonstrates removing a labeling callback.

```

pds.removeConnectionLabelingCallback( callback );

```

5.3 Integration of UCP with DRCP

Natively, DRCP supports connection tagging, which is a single label without weights. So, labeling with a single label works transparently if you use UCP with DRCP.

Note

Oracle recommends that the maximum pool size of UCP should not be bigger than the size of DRCP. If the UCP pool size is bigger than the DRCP size, then you must set the `setValidateConnectionOnBorrow` property to `off`. Otherwise, UCP keeps invalidating and closing the connections that are not associated with DRCP at that moment, and keeps creating fresh connections.

See Also

[Overview of Validating Connections in UCP](#)

Multiple label UCP connections work, but they have the following behavior changes:

- The `cost` method in the `ConnectionLabelingCallback` API is not invoked if you use UCP with DRCP using connection labeling
- UCP can invoke the `configure` method in the `ConnectionLabelingCallback` API more with DRCP configuration than without DRCP configuration.

See Also

Oracle Database JDBC Developer's Guide for more information about DRCP

5.4 Applying Connection Labels in UCP

Labels are applied on a borrowed connection using the `applyConnectionLabel` method from the `LabelableConnection` interface. This method is typically called from the `configure` method of the labeling callback. Any number of connection labels may be cumulatively applied on a borrowed connection. Each time a label is applied to a connection, the supplied key/value pair is added to the collection of labels already applied to the connection. Only the last applied value is retained for any given key.

Note

A labeling callback must be registered on the connection pool before a label can be applied on a borrowed connection; otherwise, an exception is thrown.

The following example demonstrates initializing a connection with a transaction isolation level and then applying a label to the connection:

```
String pname = "property1";
String pvalue = "value";
Connection conn = pds.getConnection();

// initialize the connection as required.

conn.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);

((LabelableConnection) conn).applyConnectionLabel(pname, pvalue);
```

In order to remove a given key from the set of connection labels applied, apply a label with the key to be removed and a null value. This may be used to clear a particular key/value pair from the set of connection labels.

Related Topics

- [Implementation of a Labeling Callback in UCP](#)

5.5 Borrowing Labeled Connections from UCP

A pool-enabled data source provides two `getConnection` methods that are used to borrow a labeled connection from the pool. The methods are shown below:

```
public Connection getConnection(java.util.Properties labels )
    throws SQLException;

public Connection getConnection( String user, String password,
                                java.util.Properties labels )
    throws SQLException;
```

The methods require that the label be passed to the `getConnection` method as a `Properties` object. The following example demonstrates getting a connection with the label `property1`, value.

```
String pname = "property1";
String pvalue = "value";
Properties label = new Properties();
label.setProperty(pname, pvalue);

Connection conn = pds.getConnection(label);
```

5.6 Checking Unmatched Labels in UCP

A connection may have multiple labels that each uniquely identifies the connection based on some desired criteria. The `getUnmatchedConnectionLabels` method is used to verify which connection labels matched from the requested labels and which did not. The method is used after a connection with multiple labels is borrowed from the connection pool and is typically used by a labeling callback. The following example demonstrates checking for unmatched labels.

```
String pname = "property1";
String pvalue = "value";
Properties label = new Properties();
label.setProperty(pname, pvalue);

Connection conn = pds.getConnection(label);
Properties unmatched = ((LabelableConnection)
    connection).getUnmatchedConnectionLabels (label);
```

5.7 Removing a Connection Label in UCP

The `removeConnectionLabel` method is used to remove a label from a connection. This method is used after a labeled connection is borrowed from the connection pool. The following example demonstrates removing a connection label.

```
String pname = "property1";
String pvalue = "value";
Properties label = new Properties();
label.setProperty(pname, pvalue);
```

```
Connection conn = pds.getConnection(label);  
((LabelableConnection) conn).removeConnectionLabel(pname);
```

6

Controlling Reclaimable Connection Behavior

This chapter describes the following interfaces:

- [AbandonedConnectionTimeoutCallback Interface](#)
- [TimeToLiveConnectionTimeoutCallback Interface](#)

6.1 AbandonedConnectionTimeoutCallback Interface

The `AbandonedConnectionTimeoutCallback` callback interface is used for the abandoned connection timeout feature. This feature enables applications to provide customized handling of abandoned connections. The callback object either uses one of its logical connection proxies or it is registered with each pooled connection. This enables applications to perform customized handling, when a particular connection is deemed abandoned by the pool. The `handleTimedOutConnection` method is invoked when a borrowed connection is deemed abandoned by the Universal Connection Pool. Applications can perform one of the following operations on the connection:

- Completely override the pool-handling process
- Invoke additional handling actions
- Assume the default pool-handling

The JDBC applications can invoke `cancel`, `close`, and `rollback` methods on the abandoned connection within the `handleTimedOutConnection` method.

Note

If you try to register more than one `AbandonedConnectionTimeoutCallback` interface on the same connection, then it results in an exception. This exception can be a `UniversalConnectionPoolException` at the pool layer or a `java.sql.SQLException`, specific to the type of the UCP Adapter like JDBC, JCA and so on.

6.2 TimeToLiveConnectionTimeoutCallback Interface

The `TimeToLiveConnectionTimeoutCallback` callback interface used for the time-to-live (TTL) connection timeout feature. This enables applications to provide customized handling for TTL timed-out connections.

The callback object either uses one of its logical connection proxies or it is registered with each pooled connection. This enables applications to perform customized handling, when the TTL of the particular connection times out.

The `handleTimedOutConnection` method is invoked when a borrowed connection is found to be TTL timed-out by the Universal Connection Pool. Applications can perform one of the following operations on the connection:

- Completely override the pool-handling process

- Invoke additional handling actions
- Assume the default pool-handling

The JDBC applications can invoke `cancel`, `close`, and `rollback` methods on the abandoned connection within the `handleTimedOutConnection` method.

 Note

If you try to register more than one `TimeToLiveConnectionTimeoutCallback` interface on the same connection, then it results in an exception. This exception can be a `UniversalConnectionPoolException` at the pool layer or a `java.sql.SQLException`, specific to the type of the UCP Adapter like JDBC, JCA, and so on.

7

Using the Connection Pool Manager

The following sections are included in this chapter:

- [Overview of Using the UCP Manager](#)
- [Overview of JMX-based Management](#)

7.1 Overview of Using the UCP Manager

The Universal Connection Pool (UCP) manager creates and maintains UCP instances. A pool instance is registered with the pool manager every time a new pool is created. This section covers the following topics:

- [About Connection Pool Manager](#)
- [Creating a Connection Pool Manager for UCP](#)
- [Life Cycle States of a Connection](#)
- [Maintenance of Universal Connection Pool](#)

7.1.1 About Connection Pool Manager

Applications use a connection pool manager to explicitly create and manage UCP JDBC connection pools. Applications use the manager because it offers full life cycle control, such as creating, starting, stopping, and destroying a connection pool. Applications also use the manager to perform routine maintenance on the connection pool, such as refreshing, recycling, and purging connections in a pool. Lastly, applications use the connection pool manager because it offers a centralized integration point for administrative tools and consoles.

7.1.2 Creating a Connection Pool Manager for UCP

A connection pool manager is an instance of the `UniversalConnectionPoolManager` interface, which is located in the `oracle.ucp.admin` package. The manager is a Singleton instance that is used to manage multiple connection pools per JVM. The interface includes methods for interacting with a connection pool manager. UCP includes an implementation that is used to get a connection pool manager instance. The following example demonstrates creating a connection pool manager instance using the implementation:

```
UniversalConnectionPoolManager mgr = UniversalConnectionPoolManagerImpl.  
getUniversalConnectionPoolManager();
```

7.1.3 Life Cycle States of a Connection

Applications use the connection pool manager to explicitly control the life cycle of connection pools. The manager is used to create, start, stop, and destroy connection pools. Life cycle methods are included as part of the `UniversalConnectionPoolManager` interface.

Understanding Life Cycle States

The life cycle states of a connection pool affects what manager operations can be performed on a connection pool. Applications that explicitly control the life cycle of a pool must ensure that the manager's operations are used only when the pool is in an appropriate state. Life cycle constraints are discussed throughout this section.

The following list describes the life cycle states of a pool:

- **Starting** : Indicates that the connection pool's start method has been called and it is in the process of starting up.
- **Running** : Indicates that the connection pool has been started and is ready to give out connections.
- **Stopping** : Indicates that the connection pool is in the process of stopping.
- **Stopped** : Indicates that the connection pool is stopped.
- **Failed** : Indicates that the connection pool has encountered failures during starting, stopping, or execution.

7.1.3.1 Creating a Connection Pool

The `CreateConnectionPool` method of the Connection Manager creates and registers a connection pool. The manager uses a connection pool adapter to create the pool and relies on a pool-enabled data source to configure the pool properties. An application must not implicitly start a connection pool before using the `createConnectionPool` method to explicitly create the same pool.

The following example demonstrates creating a connection pool instance using the manager:

```
UniversalConnectionPoolManager mgr = UniversalConnectionPoolManagerImpl.  
getUniversalConnectionPoolManager();  
  
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();  
pds.setConnectionPoolName("mgr_pool");  
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");  
pds.setURL("jdbc:oracle:thin:@//localhost:1521/XE");  
pds.setUser("<user>");  
pds.setPassword("<password>");  
  
mgr.createConnectionPool((UniversalConnectionPoolAdapter)pds);
```

An application does not have to use the manager to create a pool in order for the pool to be managed. A pool that is implicitly created (that is, automatically created when using a pool-enabled data source) and configured with a pool name, is automatically registered and managed by the pool manager. Oracle recommends implicit pool creation.

Pool Naming Convention

A connection pool name must be defined as part of the configuration. The pool name provides a way to refer to specific pools when interacting with the manager. A connection pool name must be unique and cannot be used by more than one connection pool. The manager throws a `pool already exists` exception if a connection pool already exists with the same name.

Compatibility with JBoss

JBoss users can use the JBoss-specific silent reload functionality by setting the `oracle.ucp.destroyOnReload` JVM system property to `true`. When the `oracle.ucp.destroyOnReload` property is set to `true`, then the JBoss-specific behavior automatically destroys an old pool instance prior to

creating a new one with the same name. If this system property is not set or set to false, then UCP throws a `pool already exists` exception.

7.1.3.2 Starting a Connection Pool

The manager's `startConnectionPool` method starts a connection pool using the pool name as a parameter to determine which pool to start. The pool name is defined as a pool property on a pool-enabled data source.

The following example demonstrates starting a connection pool:

```
mgr.startConnectionPool("mgr_pool");
```

An application must always create a connection pool using the manager's `createConnectionPool` method prior to starting the pool. In addition, a life cycle state exception occurs if an application attempts to start a pool that has been previously started or if the pool is in a state other than stopped or failed.

7.1.3.3 Stopping a Connection Pool

The manager's `stopConnectionPool` method stops a connection pool using the pool name as a parameter to determine which pool to stop. The pool name is defined as a pool property on the pool-enabled data source. Stopping a connection pool closes all available and borrowed connections.

The following example demonstrates stopping a connection pool:

```
mgr.stopConnectionPool("mgr_pool");
```

An application can use the manager to stop a connection pool that was started implicitly or explicitly. An error occurs if an application attempts to stop a pool that does not exist or if the pool is in a state other than started or starting.

7.1.3.4 Destroying a Connection Pool

The manager's `destroyConnectionPool` method stops a connection pool and removes it from the connection pool manager. A pool name is used as a parameter to determine which pool to destroy. The pool name is defined as a pool property on the pool-enabled data source.

The following example demonstrates destroying a connection pool:

```
mgr.destroyConnectionPool("mgr_pool");
```

An application cannot start a connection pool that has been destroyed and must explicitly create and start a new connection pool.

7.1.4 Maintenance of Universal Connection Pool

Applications use the connection pool manager to perform maintenance on a connection pool. Maintenance includes refreshing, recycling, and purging a connection pool. The maintenance methods are included as part of the `UniversalConnectionPoolManager` interface.

Maintenance is typically performed to remove and replace invalid connections and ensures a high availability of valid connections. Invalid connections typically cannot be used to connect to a database but are still maintained by the pool. These connections waste system resources and directly affect a pool's maximum connection limit. Ultimately, too many invalid connections negatively affects an applications performance.

Note

Applications can check whether or not a connection is valid when borrowing the connection from the pool. If an application consistently has a high number of invalid connections, additional testing should be performed to determine the cause.

Related Topics

- [Overview of Validating Connections in UCP](#)

7.1.4.1 Refreshing a Connection Pool

Refreshing a connection pool replaces every connection in the pool with a new connection. Any connections that are currently borrowed are marked for removal and refreshed after the connection is returned to the pool. The manager's `refreshConnectionPool` method refreshes a connection pool using the pool name as a parameter to determine which pool to refresh. The pool name is defined as a pool property on the pool-enabled data source.

The following example demonstrates refreshing a connection pool:

```
mgr.refreshConnectionPool("mgr_pool");
```

7.1.4.2 Recycling a Connection Pool

Recycling a connection pool replaces only invalid connection in the pool with a new connection and does not replace borrowed connections. The manager's `recycleConnectionPool` method recycles a connection pool using the pool name as a parameter to determine which pool to recycle. The pool name is defined as a pool property on the pool-enabled data source.

The `setSQLForValidateConnection` property must be set when using non-Oracle drivers. UCP uses this property to determine whether or not a connection is valid before recycling the connection.

The following example demonstrates recycling a connection pool:

```
mgr.recycleConnectionPool("mgr_pool");
```

Related Topics

- [Overview of Validating Connections in UCP](#)

7.1.4.3 Purging a Connection Pool

Purging a connection pool removes every connection (available and borrowed) from the connection pool and leaves the connection pool empty. Subsequent requests for a connection result in a new connection being created. The manager's `purgeConnectionPool` method purges a connection pool using the pool name as a parameter to determine which pool to purge. The pool name is defined as a pool property on the pool-enabled data source.

The following example demonstrates purging a connection pool:

```
mgr.purgeConnectionPool("mgr_pool");
```

Note

Connection pool properties, such as `minPoolSize` and `initialPoolSize`, may not be enforced after a connection pool is purged.

7.2 Overview of JMX-Based Management in UCP

JMX (Java Management Extensions) is a Java technology that supplies tools for managing and monitoring applications, system objects, devices, service-oriented networks, and JVM (Java Virtual Machine). In JMX, a given resource is instrumented by one or more Java objects known as MBeans (Managed Beans). An MBean is composed of an MBean interface and a class. The MBean interface lists the methods for all exposed attributes and operations. The class implements this interface and provides the functionality of the instrumented resource.

The MBeans are registered in a core managed object server, known as an MBean server, which acts as a management agent and can run on most devices enabled for the Java programming language. A JMX agent consists of an MBean server, in which MBeans are registered, and a set of services for handling MBeans.

See Also

- <https://docs.oracle.com/javase/tutorial/jmx/mbeans/standard.html>
- *Oracle Universal Connection Pool Java API Reference*

UCP provides the following two MBeans for pool management support:

- [UniversalConnectionPoolManagerMBean](#)
- [UniversalConnectionPoolMBean](#)

Note

All MBean attributes and operations are available only when the `UniversalConnectionPoolManager.isJmxEnabled` method returns `true`. The default value of this flag is `true`. This default value can be altered by calling the `UniversalConnectionPoolManager.setJmxEnabled` method. When an MBeanServer is not available, the `jmxFlag` is automatically set to `false`.

7.2.1 UniversalConnectionPoolManagerMBean

The `UniversalConnectionPoolManagerMBean` is a manager MBean that includes all the functionalities of a conventional connection pool manager. The `UniversalConnectionPoolManagerMBean` provides the following functionalities:

- Registering and unregistering pool MBeans
- Pool management operations like starting the pool, stopping the pool, refreshing the pool, and so on
- Starting and stopping DMS statistics

- Logging

7.2.2 UniversalConnectionPoolMBean

The `UniversalConnectionPoolMBean` is a pool MBean that covers dynamic configuration of pool properties and pool statistics. The `UniversalConnectionPoolMBean` provides the following functionalities:

- Configuring pool property attributes like size, timeouts, and so on
- Pool management operations like refreshing the pool, recycling the pool, and so on
- Monitoring pool statistics and life cycle states

8

Shared Pool Support for Multitenant Data Sources

Starting from Oracle Database 12c Release 2 (12.2.0.1), multiple data sources of multitenant data sources can share a common pool of connections in UCP and repurpose connections in the common connection pool, whenever needed. This section describes the following concepts related to new Shared Pool feature:

Note

- Only the JDBC Thin driver supports the Shared Pool feature, and not the JDBC OCI driver.
- For using this feature, you *must* use an XML configuration file.

- [Overview of Shared Pool Support](#)
- [Prerequisites for Supporting Shared Pool](#)
- [Configuring the Shared Pool](#)
- [APIs for Shared Pool Support](#)
- [Sample XML Configuration File for Shared Pool](#)

Related Topics

- [Sample XML Configuration File for Shared Pool](#)

8.1 Overview of Shared Pool Support

In UCP, the pool instances have a one-to-one mapping with the data sources. Every data source creates its own connection pool instance and that instance is not accessible or shared by another data source, even if they internally create and cache connections to the same database and service. In this architecture, a lot of isolated connection pools are created, which causes a scalability problem because a database can scale up to only a certain number of connections.

Starting from Oracle Database 12c Release 2 (12.2.0.1), UCP supports multiple data sources, connected to the same database, to share the same connection pool. This common connection pool is called as Shared Pool. The Shared Pool optimizes system resources for a scalable deployment of multitenant Java applications in Oracle Database Multitenant environment. This feature provides more flexibility in situations when there is an uneven load on each data source. When individual pool per data sources are created, then it is impossible to move around idle resources from an idle connection pool to a loaded one. However, when a Shared Pool is used, connections can be utilized in an efficient way by sharing and repurposing connections between the data sources. So, this feature reduces the total number of database connections, and improves resource usage, diagnosability, manageability, and scaling at the database servers.

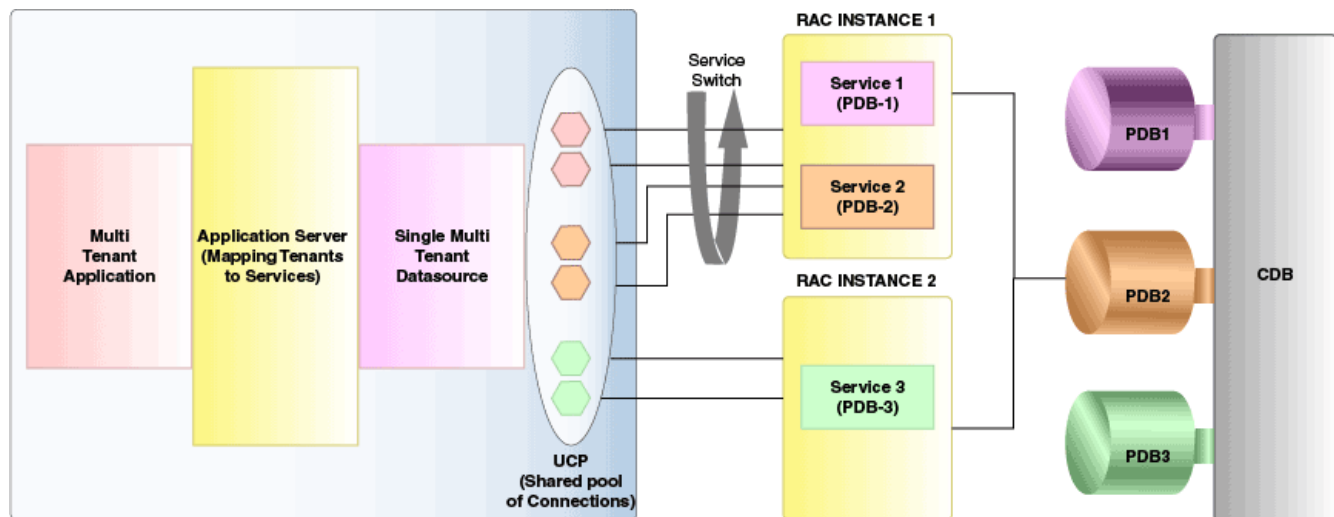
Following are the two scenarios in which this feature can be implemented:

- Single Multitenant Data Source Using Shared Pool
- One Data Source per Tenant Using Shared Pool

Single Multitenant Data Source Using Shared Pool

With this configuration, multiple tenants use the common data source and a common pool to serve connections with different services applicable to each of the tenants, as illustrated in the following diagram:

Figure 8-1 Single Multitenant Data Source Using Shared Pool



The following code snippet explains how this feature works:

```
PoolDataSource multiTenantDS = PoolDataSourceFactory.getPoolDataSource();

//common user for the CDB
multiTenantDS.setUser("c##common_user");
multiTenantDS.setPassword("password");

//Points to the root service of the CDB
multiTenantDS.setURL("jdbc:oracle:thin:@(DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)"
    + "(HOST=myhost)(PORT=5521))(CONNECT_DATA=(SERVICE_NAME=root.oracle.com))))");

// password enabled role for tenant-1
Properties tenant1Roles = new Properties();
tenant1Roles.put("tenant1-role", "tenant1-password");

//Create Connection to Tenant-1 and apply the tenant specific PDB roles.
Connection tenant1Connection =
    multiTenantDS.createConnectionBuilder()
        .serviceName("tenant1Svc.oracle.com")
        .pdbRoles(tenant1Roles)
        .build();

// password enabled role for tenant-2
```

```

Properties tenant2Roles = new Properties();
tenant1Roles.put("tenant2-role", "tenant2-password");

//Create Connection to Tenant-2 and apply the tenant specific PDB roles.
Connection tenant2Connection =
    multiTenantDS.createConnectionBuilder()
        .serviceName("tenant2Svc.oracle.com")
        .pdbRoles(tenant2Roles)
        .build();

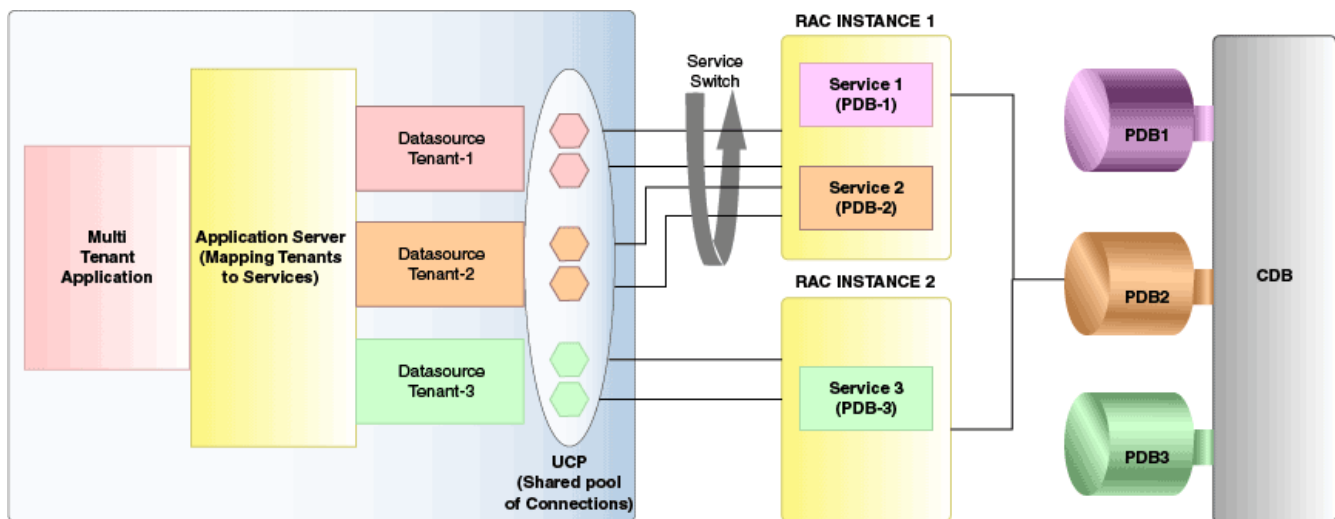
```

Refer to the “UCP APIs for Shared Pool Support” section for more information about the new APIs.

One Data Source per Tenant Using Shared Pool

With this configuration, multitenant applications have separate data sources per tenant and a common Shared Pool for connections. This results in the individual data sources being configured with tenant specific service information and sharing a common pool, as illustrated in the following diagram:

Figure 8-2 One Data Source per Tenant Using Shared Pool



The following code snippet explains how this feature works:

```

// UCP XML configuration file path in case of Unix
String file_URI = "file:/user/app/sharedpool/initial-shared-pool-config.xml";

// UCP XML configuration file path in case of Windows
String file_URI = "file:/D:/user/app/sharedpool/initial-shared-pool-config.xml";

// Java system property to specify XML configuration file location
System.setProperty("oracle.ucp.jdbc.xmlConfigFile", <file_URI>);

// Get the datasource instance, named as "pds1" in XML configuration file(initial-shared-pool-config.xml)
PoolDataSource pds1 = PoolDataSourceFactory.getPoolDataSource("pds1");
Connection pds1Conn = pds1.getConnection();

```

```
// Get the datasource instance, named as "pds2" in XML configuration file(initial-shared-pool-config.xml)
PoolDataSource pds2 = PoolDataSourceFactory.getPoolDataSource("pds2");
Connection pds2Conn = pds2.getConnection();

// Reconfigure datasource(pds1) using the new properties
Properties newProps = new Properties();
newProps.put("serviceName", <newServiceName>);
pds1.reconfigureDataSource(newProps);

// Configure a new datasource(pds3) to running pool using the new data source properties
Properties dataSourceProps = new Properties();
dataSourceProps.put("serviceName", <serviceName>);
dataSourceProps.put("connectionPoolName", <poolName>);
dataSourceProps.put("dataSourceName", <dataSourceName>);
PoolDataSource pds3 = PoolDataSourceFactory.getPoolDataSource(dataSourceProps);

// Reconfigure connection pool("pool1") using the new properties

Properties newPoolProps = new Properties();
newPoolProps.put("initialPoolSize", <newInitialPoolSizeValue>);
newPoolProps.put("maxPoolSize", <newMaxPoolSizeValue>);
UniversalConnectionPoolManager ucpMgr =
UniversalConnectionPoolManagerImpl.getUniversalConnectionPoolManager();
ucpMgr.reconfigureConnectionPool("pool1", newPoolProps);
```

① Note

- UCP uses a service switch for implementing this feature. However, the service switch in Shared Pools is supported only for homogenous services. There is no support for heterogeneous services (heterogeneity in terms of service attributes like Transaction Guard and Application Continuity) in Shared Pools.
- For the XML configuration file used in the code snippets, refer to the “XML Configuration File Required for Shared Pool Support” section.

8.2 Prerequisites for Supporting Shared Pool

Following are the prerequisites for multitenant data sources to use the Shared Pool:

- You must provide the initial configuration of Shared Pools through an XML configuration file. You can specify the initial XML configuration file for UCP through the system property `oracle.ucp.jdbc.xmlConfigFile`. The location of the initial XML configuration file should be specified as a URI. For example, `file:/user_directory/ucp.xml`.

The `configuration.xsd` schema file is included in the `ucp.jar` file for reference. Refer to this file while creating a UCP XML configuration file.

- During the reconfiguration of a shared pool, updated pool properties should be provided through reconfiguration APIs.
- Always use application service for the services used for Shared Pool, and for the individual tenant data source specific services. Connections are not repurposed or reused when an Administrative service or default PDB services are used.

- The various services accessed through the Shared Pool must be homogenous, that is, they should have similar properties with respect to Application Continuity (AC), DRCP (Database Resident Connection Pool), and so on.
- The Shared Pool must be configured with a single user, and this user should be a common user configured on the CDB. The common user should have the following privileges - CREATE SESSION, ALTER SESSION, and SET CONTAINER. The common user should also have the execute permission on the DBMS_SERVICE_PRIVT package.

Note

- If the common user needs specific roles or password-enabled roles per tenant, then these roles should be specified in the respective tenant data source properties.
- The advantage of the SET CONTAINER statement is that the pool does not have to create a new connection to a PDB, if there is an existing connection to a different PDB. The pool can use the existing connection and can connect to the desired PDB through the SET CONTAINER statement.

- Connection repurposing among various tenant connections in the Shared Pool happens only when the total number of the connections in the pool reaches the connection repurpose threshold (if configured on the pool) and the minimum pool size.
- The URL specified for the Shared Pool in the XML configuration file must have the LONG format, with service name explicitly specified. Short format or Easy Connection URL is not supported.

8.3 Configuring the Shared Pool

The following sections describe the Shared Pool configuration:

- Initial Configuration of the Pool
- Reconfiguration of the Pool

Initial Configuration of the Pool

For the initial configuration of the pool, get a data source instance by using the XML configuration file and then, using that data source, get a connection from a Shared Pool.

```
// Get the data source instance, named as "pds1" in the XML configuration file(initial-shared-pool-config.xml)
PoolDataSource pds1 = PoolDataSourceFactory.getPoolDataSource("pds1");
Connection pds1Conn = pds1.getConnection();
```

Reconfiguration of the Pool

- The following code snippet shows how to reconfigure the data source that you obtained during the initial configuration of the pool:

```
// Reconfigure datasource(pds1) using the new properties for reconfiguration
```

```
Properties newProps = new Properties();
newProps.put("serviceName", <newServiceName>);
pds1.reconfigureDataSource(newProps);
```

- The following code snippet shows how to add a new data source to an already running Shared Pool:

```
// Configure a new datasource(pds3) to the running pool using the new data source properties
```

```
Properties dataSourceProps = new Properties();
dataSourceProps.put("serviceName", <serviceName>);
dataSourceProps.put("connectionPoolName", <poolName>);
dataSourceProps.put("dataSourceName", <dataSourceName>);
PoolDataSource pds3 = PoolDataSourceFactory.getPoolDataSource(dataSourceProps);
```

- The following code snippet shows how to reconfigure the connection pool:

```
// Reconfigure connection pool("pool1") using the new properties
```

```
Properties newPoolProps = new Properties();
newPoolProps.put("initialPoolSize", <newInitialPoolSizeValue>);
newPoolProps.put("maxPoolSize", <newMaxPoolSizeValue>);
UniversalConnectionPoolManager ucpMgr =
UniversalConnectionPoolManagerImpl.getUniversalConnectionPoolManager();
ucpMgr.reconfigureConnectionPool("pool1", newPoolProps);
```

8.4 UCP APIs for Shared Pool Support

New Methods in PoolDataSource Interface

The following methods have been introduced in the `oracle.ucp.jdbc.PoolDataSource` interface:

- `reconfigureDataSource(Properties configuration)`
- `getMaxConnectionsPerService()`
- `getServiceName()`
- `getPdbRoles()`
- `getConnectionRepurposeThreshold()`
- `setConnectionRepurposeThreshold(int threshold)`

New Methods in PoolDataSourceFactory Class

The following methods have been introduced in the `oracle.ucp.jdbc.PoolDataSourceFactory` class:

- `getPoolDataSource(String dataSourceName)`
- `getPoolDataSource(Properties configuration)`
- `getPoolXADataSource(String dataSourceName)`
- `getPoolXADataSource(Properties configuration)`

New Method in oracle.ucp.admin.UniversalConnectionPoolManager Interface

The following method has been introduced in the `oracle.ucp.admin.UniversalConnectionPoolManager` interface:

```
reconfigureConnectionPool(String poolName , Properties configuration)
```

New Method in oracle.ucp.admin.UniversalConnectionPool Interface

The following method has been introduced in the oracle.ucp.admin.UniversalConnectionPool interface:

- isShareable()
- getMaxConnectionsPerService()
- setMaxConnectionsPerService(int maxConnectionsPerService)

① See Also

Oracle Universal Connection Pool Java API Reference for more information about these methods.

8.5 Sample XML Configuration File for Shared Pool

initial-shared-pool-config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<ucp-properties>
  <connection-pool
    connection-pool-name="pool1"
    connection-factory-class-name="oracle.jdbc.pool.OracleDataSource"
    url="jdbc:oracle:thin:@(DESCRIPTION=(ADDRESS=(HOST=host_name)(PORT=1521)(PROTOCOL=tcp))
(CONNECT_DATA=(SERVICE_NAME=myorclpdbservice)))"
    user="C##CommonUser"
    password=password
    initial-pool-size="10"
    min-pool-size="5"
    max-pool-size="20"
    connection-repurpose-threshold="13"
    max-connections-per-service="15"
    validate-connection-on-borrow="true"
    sql-for-validate-connection="select 1 from dual"
    shared="true"
  >

  <connection-property name="oracle.jdbc.ReadTimeout" value="2000"/>
  <connection-property name="oracle.net.OUTBOUND_CONNECT_TIMEOUT" value="2000"/>

  <data-source
    data-source-name="pds1"
    service=pdb1_service_name
    description="pdb1 data source"/>

  <data-source
    data-source-name="pds2"
    service=pdb2_service_name
    description="pdb2 data source"/>
```

```
</connection-pool>  
</ucp-properties>
```

9

Using Oracle RAC Features

The following sections are included in this chapter:

- [Overview of Oracle RAC Features](#)
- [About Fast Connection Failover](#)
- [About Run-Time Connection Load Balancing](#)
- [About Connection Affinity](#)
- [Global Data Services](#)

9.1 Overview of Oracle RAC Features

UCP JDBC connection pools provide a tight integration with various Oracle Real Application Clusters (Oracle RAC) Database features. The features include Fast Connection Failover (FCF), Run-Time Connection Load Balancing, and Connection Affinity. These features require the use of an Oracle JDBC driver, Oracle RAC database, and the Oracle Notification Service library (ons.jar) that is included with the Oracle Client software.

Applications use Oracle RAC features to maximize connection performance and availability and to mitigate down-time due to connection problems. Applications have different availability and performance requirements and should implement Oracle RAC features accordingly.

Note

Starting from Oracle Database 11g Release 1 (11.2), FCF is also supported by Oracle Restart on a single instance database. Oracle Restart was previously known as Single-Instance High Availability (SIHA).

See Also

- *Oracle Real Application Clusters Administration and Deployment Guide* for more information about these technologies
- *Oracle Database Administrator's Guide* for more information about Oracle Restart

Generic High Availability and Performance Features

The UCP APIs and connection pool properties include many high availability and performance features that do not require an Oracle RAC database. These features work well with both Oracle and non-Oracle connections and are discussed throughout this guide. For example: validating connections on borrow; setting timeout properties; setting maximum reuse properties; and connection pool manager operations are all used to ensure a high-level of connection availability and optimal performance.

Note

Generic high availability and performance features work slightly better when using Oracle connections because UCP leverages Oracle JDBC internal APIs.

Database Version Compatibility for Oracle RAC

The following table lists supported Database versions for various Oracle RAC features:

Table 9-1 Oracle RAC Version Compatibility

Feature	Supported Database Version
Fast Connection Failover	Oracle Database 10.1.x and later versions
Run-time Connection Load-Balancing	Oracle Database 10.2.x and later versions
Web Session Affinity	Oracle Database 11.1.x and later versions
Transaction-Based Affinity	Oracle Database 10.2.x and later versions (Oracle Database 11.1.x recommended)

Oracle JDBC Driver Version Compatibility for Oracle RAC

Oracle JDBC driver 10.1.x and later versions are supported with Oracle RAC features.

9.2 About Fast Connection Failover

This section contains the following subsections:

- [Overview of Fast Connection Failover](#)
- [What is Fast Connection Failover](#)
- [Fast Connection Failover Prerequisites](#)
- [Example of Fast Connection Failover Configuration](#)
- [Enabling Fast Connection Failover](#)
- [What is ONS](#)
- [Configuring the Connection URL](#)

9.2.1 Overview of Fast Connection Failover

The Fast Connection Failover (FCF) feature is a Fast Application Notification (FAN) client implemented through the connection pool. The feature requires the use of an Oracle JDBC driver and high availability (HA) database configurations like Oracle RAC, Single Restart, or Active Data Guard.

Note

This section describes only the steps that an application must perform when using FCF with Oracle RAC.

① See Also

Oracle Real Application Clusters Administration and Deployment Guide for more information

FCF is useful in the following scenarios:

- **Unplanned Outages:** FCF rapidly detects dead connections,, and then terminates and removes them from the pool. Connection removal relies on terminating the sever-socket connections rapidly to prevent the system from becoming nonresponsive. Borrowed and in-use connections are interrupted only for unplanned outages.
- **Planned Outages:** FCF does not interrupt and close the borrowed or in-use connections until work is done and control of the connection is returned to the pool.
- **Fatal Connection Errors and Exceptions:** FCF encapsulates fatal connection errors and exceptions into the `isValid` API for robust and efficient retries.
- **Addition of New Nodes to the Cluster:** FCF recognizes the new nodes that join an Oracle RAC cluster and places new connections on that node appropriately for delivering maximum quality of service to applications at run time. This facilitates middle-tier integration of Oracle RAC node joins and work-request routing from the application tier.
- **Run-Time Work Requests:** FCF distributes run-time work requests to all active Oracle RAC instances.

Unplanned Shutdown Scenarios

FCF supports unplanned shutdown scenarios by detecting and removing stale connections to an Oracle RAC cluster. Stale connections include connections that do not have a service available on any instance in an Oracle RAC cluster due to service-down and node-down events. Borrowed connections and available-but-stale connections are detected, and their network connection is severed before removing them from the pool. These removed connections are not replaced by the pool. Instead, the application must retry connections before performing any work with a connection.

① Note

Borrowed connections are immediately terminated and closed during unplanned shutdown scenarios. Any on-going transactions immediately receive an exception.

Planned Shutdown Scenarios

FCF supports planned shutdown scenarios where an Oracle RAC service can be gracefully shutdown. In such scenarios, stale borrowed connections are marked, and are terminated and removed after they are returned to the pool. Any on-going transactions do not see any difference and proceed to complete.

The primary difference between unplanned and planned shutdown scenarios is how borrowed connections are handled. Stale connections that are idle in the pool (not borrowed) are removed in the same manner as the unplanned shutdown scenario.

UCP also supports graceful connection draining from any planned-down Oracle RAC instance. Affected borrowed connections are removed smoothly over a grace period, instead of immediate removal upon their return to the pool. This helps in avoiding throughput impact and logon storms during any service relocation.

Oracle RAC Instance Rejoin and New Instance Scenarios

FCF supports scenarios where an Oracle RAC cluster adds instances that provide a service of interest. The instance may be new to the cluster or may have been restarted after a down event. In both cases, UCP recognizes the new instance and creates connections to the node as required.

Related Topics

- [Checking If a Connection Is Valid](#)
- [Enabling Fast Connection Failover](#)

9.2.2 What is Fast Connection Failover?

After Fast Connection Failover is enabled, the mechanism is automatic; no application intervention is needed. This section discusses how a connection failover is presented to an application and what steps the application takes to recover, in the following sections:

- [What the Application Sees](#)
- [How FCF Works](#)

9.2.2.1 What the Application Sees

By the time an Oracle RAC service failure is propagated to the JDBC application, the database already rolls back the local transaction. The cache manager then cleans up all invalid connections. When an application holding an invalid connection tries to do work through that connection, it is possible to receive `SQLException`, `ORA-17008, Closed Connection`.

When an application receives a `Closed Connection` error message, it should do the following:

1. Retry the connection request. This is essential, because the old connection is no longer open.
2. Replay the transaction. All work done before the connection was closed has been lost.

Note

The application should not try to roll back the transaction. The transaction was already rolled back in the database by the time the application received the exception.

9.2.2.2 How FCF Works

Under Fast Connection Failover, each connection in the cache maintains a mapping to a service, instance, database, and host name.

When a database generates an Oracle RAC event, that event is forwarded to the JVM in which JDBC is running. A daemon thread inside the JVM receives the Oracle RAC event and passes it on to the Connection Cache Manager. The Connection Cache Manager then throws `SQL` exceptions to the applications affected by the Oracle RAC event.

A typical failover scenario may work like the following:

1. A database instance fails, leaving several stale connections in the cache.

2. The Oracle RAC mechanism in the database generates an Oracle RAC event which is sent to the JVM containing JDBC.
3. The daemon thread inside the JVM finds all the connections affected by the Oracle RAC event, notifies them of the closed connection through SQL exceptions, and rolls back any open transactions.
4. Each individual connection receives a SQL exception and must retry.

9.2.3 Fast Connection Failover Prerequisites

Fast Connection Failover is available under the following circumstances:

- The Universal Connection Pool is enabled.
Fast Connection Failover works in conjunction with the JDBC connection caching mechanism. This helps applications manage connections to ensure high availability.
- The application uses service names to connect to the database.
The application cannot use service identifiers.
- The underlying database has Oracle Database 12c Release 1 (12.1) or later Real Application Clusters (Oracle RAC) capability or Oracle Data Guard configured with either single instance Databases or Oracle RAC.
If failover events are not propagated, then connection failover cannot occur.
- Oracle Notification Service (ONS) is configured and available on the node where JDBC is running.
JDBC depends on ONS to propagate database events and notify JDBC of them.
- The Java Virtual Machine (JVM) in which your JDBC instance is running must have `oracle.ons.oraclehome` set to point to your `ORACLE_HOME`.

9.2.4 Example of Fast Connection Failover Configuration

The following example demonstrates a connection pool that uses the FCF feature. FCF is configured through a pool-enabled data source. The example includes enabling FCF, configuring the Oracle Notification Service (ONS) and configuring a connection URL. These topics are discussed after the example.

The `isValid` method of the `oracle.ucp.jdbc.ValidConnection` interface is typically used in conjunction with the FCF feature and is used to check if a borrowed connection is still usable after an SQL exception has been thrown due to an Oracle RAC down event. For example:

```
try { conn = pds.getConnection(); ... } catch (SQLException sqlexc)
{
    if (conn == null || !((ValidConnection) conn).isValid())

        // take the appropriate action

    ...
    conn.close();
}
```

Example 9-1 Fast Connection Failover Configuration Example

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionPoolName("FCFSamplePool");
pds.setFastConnectionFailoverEnabled(true);
```

```
pds.setONSConfiguration("nodes=racnode1:4200,racnode2:4200\nwalletfile=
/oracle11/onswalletfile");
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
pds.setURL("jdbc:oracle:thin@(DESCRIPTION= "+
"(LOAD_BALANCE=on)" +
"(ADDRESS=(PROTOCOL=TCP)(HOST=racnode1) (PORT=1521))"+
"(ADDRESS=(PROTOCOL=TCP)(HOST=racnode2) (PORT=1521))"+
"(CONNECT_DATA=(SERVICE_NAME=service_name)))");
...
```

Related Topics

- [Checking If a Connection Is Valid](#)

9.2.5 Enabling Fast Connection Failover

The FCF pool property is used to enable and disable FCF. FCF is disabled by default. The following example demonstrates enabling FCF as shown in [Example 9-1](#).

```
pds.setFastConnectionFailoverEnabled(true);
```

Note

Starting from Oracle Database 12c Release 1 (12.1.0.2), UCP supports the `oracle.ucp.PlannedDrainingPeriod` system property. It specifies the grace time period (in integer seconds) over which the pool smoothly drains the borrowed connections affected by a planned shut down. Draining starts when the same Database service becomes available on another instance different from the one that is going down.

When this property is not set, or set to 0, then the pool closes any affected borrowed connection immediately when it is returned to the pool.

Querying Fast Connection Failover Status

An application determines if Fast Connection Failover is enabled by calling `OracleDataSource.getFastConnectionFailoverEnabled`, which returns `true` if failover is enabled, `false` otherwise.

Note

FCF must also be enabled to use run-time connection load balancing and connection affinity. These features are discussed later in this chapter.

9.2.6 What is ONS?

FCF relies on the Oracle Notification Service (ONS) to propagate database events between the connection pool and the Oracle RAC database. At run time, the connection pool must be able to setup an ONS environment. ONS (`ons.jar`) is included as part of the Oracle Client software. ONS can be configured using either remote configuration or client-side ONS daemon configuration. Remote configuration is the preferred configuration for standalone client applications. This section discusses the following topics:

- [Overview of ONS Configuration File](#)
- [Remote Configuration of ONS](#)
- [Configuration of Client-Side ONS Daemon](#)

9.2.6.1 Overview of ONS Configuration File

ONS configuration is controlled by the ONS configuration file, `ORACLE_HOME/opmn/conf/ons.config`. This file tells the ONS daemon how it should behave. Configuration information within `ons.config` is defined in simple name and value pairs.

Some parameters in the `ons.config` file are required and some are optional. [Table 9-2](#) lists the required ONS configuration parameters and [Table 9-3](#) lists the optional ONS configuration parameters. ONS must be refreshed after updating the `ons.config` file.

Table 9-2 Required ONS Configuration Parameters

Parameter	Explanation
localport	Specifies the port that ONS binds to on the local host interface to talk to local clients. For example, <code>localport=4100</code>
remoteport	Specifies the port that ONS binds to on all interfaces for talking to other ONS daemons. For example, <code>remoteport=4200</code>
nodes	Specifies a list of other ONS daemons to talk to. Node values are given as a comma-delimited list of either host names or IP addresses plus ports. The port value that is given is the remote port that each ONS instance is listening on. In order to maintain an identical file on all nodes, the <code>host:port</code> of the current ONS node can also be listed in the nodes list. It will be ignored when reading the list. For example, <code>nodes=myhost.example.com:4200,123.123.123.123:4200</code> The nodes listed in the nodes line correspond to the individual nodes in the Oracle RAC instance. Listing the nodes ensures that the middle-tier node can communicate with the Oracle RAC nodes. At least one middle-tier node and one node in the Oracle RAC instance must be configured to see one another. As long as one node on each side is aware of the other, all nodes are visible. You need not list every single cluster and middle-tier node in the ONS configuration file of each Oracle RAC node. In particular, if one ONS configuration file cluster node is aware of the middle tier, then all nodes in the cluster are aware of it.

Table 9-3 Optional ONS Configuration Parameters

Parameter	Description
logcomp	Specifies the ONS components to log. The format is as follows: <pre><component>[<subcomponent>,...];<component>[<subcomponent>,...];...</pre> If no subcomponents need to be specified, then do not include the brackets ([]) after the component name. To exclude messages from a subcomponent, precede the subcomponent name with an exclamation mark (!). For example, to exclude messages from the topology subcomponent, you use the following format: <pre>[all,!topology]</pre> Note that before specifying a subcomponent from which you want to exclude messages, you must first ensure that the subcomponent includes the messages. Following are the valid values for components: • internal • ons If you specify the component as internal, then there are no valid values for subcomponent. If you specify the component as ons, then you can specify the following values for subcomponent: • all: Specifies all messages • ons: ONS local information • listener: ONS listener information • discover: ONS discover (server or multicast) information • servers: ONS remote servers currently up and connected to the cluster • topology: ONS current cluster wide server connection topology • server: ONS remote server connection information • client: ONS client connection information • connect: ONS generic connection information • subscribe: ONS client subscription information • message: ONS notification receiving and processing information • deliver: ONS notification delivery information • special: ONS special notification processing • internal: ONS internal resource information • secure: ONS SSL operation information • workers: ONS worker threads The following example shows that you want to log messages for all the subcomponents under ons, except the secure subcomponent: <pre>logcomp=ons[all,!secure]</pre>
logfile	Specifies a log file that ONS should use for logging messages. The default value for log file is \$ORACLE_HOME/opmn/logs/ons.log. For example, logfile=/private/oraclehome/opmn/logs/myons.log
walletfile	Specifies the wallet file used by the Oracle Secure Sockets Layer (SSL) to store SSL certificates. If a wallet file is specified to ONS, then it uses SSL when communicating with other ONS instances and require SSL certificate authentication from all ONS instances that try to connect to it. This means that if you want to turn on SSL for one ONS instance, then you must turn it on for all instances that are connected. This value should point to the directory where your ewallet.p12 file is located. For example, walletfile=/private/oraclehome/opmn/conf/ssl.wlt/default

Table 9-3 (Cont.) Optional ONS Configuration Parameters

Parameter	Description
useocr	Specifies the value, reserved for use on the server-side, to indicate ONS whether it should store all Oracle RAC nodes and port numbers in Oracle Cluster Registry (OCR) instead of the ONS configuration file or not. A value of useocr=on is used to store all Oracle RAC nodes and port numbers in Oracle Cluster Registry (OCR). Do not use this option on the client-side.
allowgroup	Specifies the ONS setting to indicate the user group connecting to the localport. When set to true, ONS allows users within the same OS group to connect to its local port. When set to false, ONS only allows the same user running the ONS daemon to access its local port. The default value of this parameter is false. When using remote ONS configuration, there is no need to set this parameter.

The `ons.config` file allows blank lines and comments on lines that begin with the number sign (#).

9.2.6.2 Remote Configuration of ONS

UCP supports remote configuration of ONS through the `ONSConfiguration` pool property. The `ONSConfiguration` pool property value is a string that closely resembles the content of the `ons.config` file. The string contains a list of `name=value` pairs separated by a new line character (`\n`). You can set this pool property in the following two ways:

- The name can be one of the following: `nodes`, `walletfile`, or `walletpassword`. The parameter string should at least specify the ONS configuration `nodes` attribute as a list of `host:port` pairs separated by a comma. SSL is used when the `walletfile` attribute is specified as an Oracle wallet file.

The following example demonstrates an ONS configuration string as shown in [Example 9-1](#):

```
...
pds.setONSConfiguration("nodes=racnode1:4200,racnode2:4200\nwalletfile=/oracle11/onswalletfile");
...
```

- The name can be only `propertiesfile`. The value is the location of an ONS-specific Java properties file. This file must contain the `oracle.ons.nodes` property, and one or both of the following ONS Java properties:

- `oracle.ons.walletfile`
- `oracle.ons.walletpassword`

The following example illustrates such an `ONSConfiguration` string:

```
pds.setONSConfiguration("propertiesfile=/usr/ons/ons.properties");
```

The following is an example of the content of the Java properties `ons.properties` file:

```
oracle.ons.nodes=racnode1:4200,racnode2:4200
oracle.ons.walletfile=/oracle11/onswalletfile
```

Note

The parameters in the configuration string must match those for the Oracle RAC Database. In addition, if you are using Oracle Application Server, then you must configure ONS using procedures that are applicable to the server.

For standalone Java applications, you must configure ONS using the `setONSConfiguration` method. However, if your application meets the following requirements, then you no longer need to call the `setONSConfiguration` method for enabling FCF:

- Your application is using Oracle Database 12c Release 1 (12.1) or later UCP and Oracle RAC Database 12c Release 1 (12.1) or later
- Your application does not require ONS wallet or keystore

9.2.6.3 Configuration of Client-Side ONS Daemon

Client-side ONS daemon configuration is typical of applications that run on a middle-tier server such as the Oracle Application Server. Clients in this scenario directly configure ONS by updating the `ons.config` file. The location of the file may be different depending on the platform.

[Example 9-2](#) demonstrates an `ons.config` file for [Example 9-1](#):

Note

For client-side ONS daemon configuration, if the operating system (OS) user that starts the connection pool and the OS user that starts the client-side daemon are different, then they both *must* belong to the same OS group. Also, the value of the `allowgroup` parameter must be set to `true` in the `ons.config` file.

After configuring ONS, you start the ONS daemon with the `onsctl` command. You *must* make sure that an ONS daemon is running at all times.

Using the `onsctl` Command

After configuring, use `ORACLE_HOME/opmn/bin/onsctl` to start, stop, reconfigure, and monitor the ONS daemon. [Table 9-4](#) is a summary of the commands that `onsctl` supports.

Table 9-4 `onsctl` Commands

Command	Effect	Output
start	Starts the ONS daemon	onsctl: ons started
stop	Stops the ONS daemon	onsctl: shutting down ons daemon...
ping	Verifies whether or not the ONS daemon is running	ons is running ...
reconfig	Triggers a reload of the ONS configuration without shutting down the ONS daemon	
help	Prints a help summary message for <code>onsctl</code>	

Table 9-4 (Cont.) onscctl Commands

Command	Effect	Output
detailed	Prints a detailed help message for onscctl	

See Also

Oracle Real Application Clusters Administration and Deployment Guide

Note

- The Java Virtual Machine (JVM), in which your JDBC instance is running, must have the `oracle.ons.oraclehome` system property set to the location of `ORACLE_HOME` before starting the application. For example:

```
java -Doracle.ons.oraclehome=$ORACLE_HOME ...
```

- Oracle recommends remote configuration of ONS for UCP.

Note

In Oracle RAC 12.1.0.2.0, by default, server installation requires the value of the `walletfile` ONS parameter to be set, and enforces the use of SSL for all ONS connections.

If you have a UCP application that is already using the `walletfile` parameter in the ONS remote configuration string or local configuration file, then the only requirement is that, for the same topology, the wallet file on the client side must have the same content as the wallet file on the server side. You can make a copy of the server-side file and make it available on the client side.

For UCP applications that are using Oracle RAC features without setting the `walletfile` parameter, you must perform one of the following:

- Add the `walletfile` parameter setting to the ONS remote configuration string or local configuration file, as shown in [Example 9-1](#). Keep in mind that, for the same topology, the wallet file on the client side must have the same content as the wallet file on the Oracle RAC server side.
- Run the following command to remove the `walletfile` parameter setting from both client and server ONS configuration string and the local configuration file:

```
srvctl modify nodeapps -clientdata
```

For secure communication, the ONS auto-configuration in Oracle RAC 12.1.x no longer works when Oracle RAC 12.1.0.2.0 is first installed or patched. Applications have to use explicit ONS configuration (remote or local) instead, and make one of the changes previously discussed.

Example 9-2 Example of a Sample ons.config File

```
# This is an example ons.config file
#
# The first three values are required
localport=4100
remoteport=4200
nodes=racnode1.example.com:4200,racnode2.example.com:4200
```

9.2.7 Configuring the Connection URL

The connection URL of a connection factory must use the service name syntax when using FCF. The service name is used to map the connection pool to the service. In addition, the factory class must be an Oracle factory class. The following example demonstrates configuring the connection URL as shown in [Example 9-1](#):

```
...
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
pds.setURL("jdbc:oracle:thin@//host:port/service_name");
...
```

Note

An exception is thrown if a service identifier (SID) is specified for the connection URL when FCF is enabled.

The following examples demonstrate valid connection URL syntax when connecting to an Oracle RAC database. Examples for both the Oracle JDBC thin and Oracle OCI driver are included. Notice that the URL can be used to explicitly enable load balancing among Oracle RAC nodes:

Valid Connection URL Usage

```
pds.setURL("jdbc:oracle:thin@//host:port/service_name");

pds.setURL("jdbc:oracle:thin@//cluster-alias:port/service_name");

pds.setURL("jdbc:oracle:thin:@(DESCRIPTION= "+
  "(LOAD_BALANCE=on)"+
  "(ADDRESS=(PROTOCOL=TCP)(HOST=host1)(PORT=1521))"+
  "(ADDRESS=(PROTOCOL=TCP)(HOST=host2)(PORT=1521))"+
  "(CONNECT_DATA=(SERVICE_NAME=service_name)))");

pds.setURL("jdbc:oracle:thin:@(DESCRIPTION= "+
  "(ADDRESS=(PROTOCOL=TCP)(HOST=cluster_alias) (PORT=1521)) "+
  "(CONNECT_DATA=(SERVICE_NAME=service_name)))");

pds.setURL("jdbc:oracle:oci:@TNS_ALIAS");

pds.setURL("jdbc:oracle:oci:@(DESCRIPTION= "+
  "(LOAD_BALANCE=on) "+
  "(ADDRESS=(PROTOCOL=TCP)(HOST=host1) (PORT=1521)) "+
  "(ADDRESS=(PROTOCOL=TCP)(HOST=host2)(PORT=1521)) "+
  "(CONNECT_DATA=(SERVICE_NAME=service_name)))");

pds.setURL("jdbc:oracle:oci:@(DESCRIPTION= "+
  "(ADDRESS=(PROTOCOL=TCP)(HOST=cluster_alias) (PORT=1521)) "+
  "(CONNECT_DATA=(SERVICE_NAME=service_name)))");
```

9.3 About Run-Time Connection Load Balancing

This section contains the following subsections:

- [Overview of Run-Time Connection Load Balancing](#)
- [Setting Up Run-Time Connection Load Balancing](#)

9.3.1 Overview of Run-Time Connection Load Balancing

In an Oracle Real Application Clusters environment, a connection could belong to any instance that provides the relevant service. In the best case, all instances perform equally well and randomly retrieving a connection from the cache is appropriate. However, when one instance performs better than others, random selection of a connection is inefficient. The run-time connection load balancing feature enables routing of work requests to an instance that offers the best performance, minimizing the need to relocate work.

UCP JDBC connection pools leverage the load balancing functionality provided by an Oracle RAC database. Run-time connection load balancing requires the use of an Oracle JDBC driver and an Oracle RAC database.

See Also

Oracle Real Application Clusters Administration and Deployment Guide

Run-time connection load balancing is useful when:

- Traditional balancing of workload is not optimal
- Requests must be routed to make optimal use of resources in a clustered database
- Capacity within the cluster differs and is expected to change over time
- The need to avoid sending work to slow, hung, and dead nodes is required

UCP uses the Oracle RAC Load Balancing Advisory. The advisory is used to balance work across Oracle RAC instances and is used to determine which instances offer the best performance. Applications transparently receive connections from instances that offer the best performance. Connection requests are quickly diverted from instances that have slowed, are not responding, or that have failed.

Run-time connection load balancing provides the following benefits:

- Manages pooled connections for high performance and scalability
- Receives continuous recommendations on the percentage of work to route to database instances
- Adjusts distribution of work based on different back-end node capacities such as CPU capacity or response time
- Reacts quickly to changes in cluster reconfiguration, application workload, overworked nodes, or hangs
- Receives metrics from the Oracle RAC Load Balance Advisory. Connections to well performing instances are used most often. New and unused connections to under-performing instances will gravitate away over time. When distribution metrics are not received, connection are selected using a random choice.

9.3.2 Setting Up Run-Time Connection Load Balancing

Run-time connection load balancing requires that FCF is enabled and configured properly.

In addition, you must configure the Oracle RAC Load Balancing Advisory with service-level goals for each service for which load balancing is enabled:

- The service goal must be set to one of the following:

- DBMS_SERVICE.SERVICE_TIME
- DBMS_SERVICE.THROUGHPUT

The service goal can be set using the `goal` parameter, and the connection balancing goal can be set using the `clb_goal` parameter.

- The connection balancing goal must be set to SHORT. For example,

```
EXECUTE DBMS_SERVICE.MODIFY_SERVICE (service_name => 'sjob' -, goal =>  
    DBMS_SERVICE.GOAL_THROUGHPUT -, clb_goal => DBMS_SERVICE.CLB_GOAL_SHORT);
```

Or

```
EXECUTE DBMS_SERVICE.MODIFY_SERVICE (service_name => 'sjob' -, goal =>  
    DBMS_SERVICE.GOAL_SERVICE_TIME -, clb_goal => DBMS_SERVICE.CLB_GOAL_SHORT);
```

The connection balancing goal can also be set by calling the `DBMS_SERVICE.create_service` procedure.

Note

You can set the connection balancing goal to LONG. However, this is mostly useful for closed workloads, that is, when the rate of completing work is equal to the rate of starting new work.

Related Topics

- [About Fast Connection Failover](#)

See Also

Oracle Real Application Clusters Administration and Deployment Guide

9.4 About Connection Affinity

This section contains the following subsections:

- [Overview of Connection Affinity](#)
- [Setting Up Connection Affinity](#)

9.4.1 Overview of Connection Affinity

UCP JDBC connection pools leverage affinity functionality provided by an Oracle RAC database. Connection affinity requires the use of an Oracle JDBC driver and an Oracle RAC database version 11.1.0.6 or higher.

Connection affinity is a performance feature that enables a connection pool to select connections that are directed at a specific Oracle RAC instance. The pool uses run-time connection load balancing (if configured) to select an Oracle RAC instance to create the first connection and then subsequent connections are created with an affinity to the same instance.

See Also

- ["Strict Affinity Mode"](#)
- *Oracle Real Application Clusters Administration and Deployment Guide* for more information about setting up an Oracle RAC database.

UCP JDBC connection pools support the following three types of connection affinity:

- [Transaction-Based Affinity](#)
- [Web Session Affinity](#)
- [Oracle RAC Data Affinity](#)

9.4.1.1 Transaction-Based Affinity

Transaction-based affinity is an affinity to an Oracle RAC instance that can be released by either the client application or a failure event. Applications typically use this type of affinity when long-lived affinity to an Oracle RAC instance is desired or when the cost (in terms of performance) of being redirected to a new Oracle RAC instance is high. Distributed transactions are a good example of transaction-based affinity. XA connections that are enlisted in a distributed transaction keep an affinity to the Oracle RAC instance for the duration of the transaction. In this case, an application would incur a significant performance cost if a connection is redirected to a different Oracle RAC instance during the distributed transaction.

9.4.1.2 Web Session Affinity

Web session affinity is an affinity to an Oracle RAC instance that can be released by either the instance, a client application, or a failure event. The Oracle RAC instance uses a hint to communicate to a connection pool whether affinity has been enabled or disabled on the instance. An Oracle RAC instance may disable affinity based on many factors, such as performance or load. If an Oracle RAC instance can no longer support affinity, the connections in the pool are refreshed to use a new instance and affinity is established once again.

Applications typically use this type of affinity when short-lived affinity to an Oracle RAC instance is expected or if the cost (in terms of performance) of being redirected to a new Oracle RAC instance is minimal. For example, a mail client session might use Web session affinity to an Oracle RAC instance to increase performance and is relatively unaffected if a connection is redirected to a different instance.

9.4.1.3 Oracle RAC Data Affinity

Data affinity describes the concept of ensuring that a group of related cache entries is contained within a single cache partition.

Starting from Oracle Database Release 18c, UCP supports Oracle RAC Data Affinity. When you enable Data Affinity on the Oracle RAC database, data on the affinized tables are partitioned in such a way that a particular partition or subset of rows for a table is affinized to a particular Oracle RAC database instance. The affinity leads to higher performance and scalability for the applications due to improved cache locality and reduced internode synchronization and block pings among the RAC instances.

① See Also

[Enabling a Custom Partition Assignment Strategy](#)

To use the Oracle RAC Data Affinity feature, the clients accessing the database through UCP must provide the data affinity key in their connection requests. UCP has the following capabilities when pooling connections for an affinity enabled RAC database:

1. UCP learns the topology that contains the data affinity of the data partitions across Oracle RAC instances at pool start up.
2. UCP connection requests that need to leverage the Oracle RAC Data Affinity feature provides the data affinity key using the sharding key builder and use the connection builder as follows:

```
PoolDataSource pds = new PoolDataSourceImpl();
// configure the datasource with the database connection properties

/* Builds the RAC data affinity key using the sharding key builder API
and gets a connection from the pool using UCP connection builder */
OracleShardingKey dataAffinityKey = pds.createShardingKeyBuilder()
    .subkey(1000, OracleType.NUMBER)
    .build();

Connection connection = pds.createConnectionBuilder()
    .shardingKey(dataAffinityKey)
    .build();
```

① Note

You can still make connection requests to Oracle RAC Data Affinity-enabled without providing the data affinity key. However, in this case, you will not see the benefits of Oracle RAC Data Affinity feature.

3. UCP determines the affinized instance for the shard key provided in the request and checks if a connection for that instance exists in the pool. If the connection exists, then it is used to serve the request. If a matching connection does not exist in the pool, then a fallback to Run-Time Load Balancing chooses a connection for the request and serves it. If a new connection needs to be created to serve the request, then the request is routed to the affinized instance corresponding to the provided shard (data affinity) key.

4. UCP keeps its topology of the data partitions in sync with the server side when there are HA events or when there is a change in the affinity of data partitions on Oracle RAC.

9.4.2 Setting Up Connection Affinity

Perform the following steps to set up connection affinity:

- Enable FCF.

① See Also

["About Fast Connection Failover"](#)

- Enable run-time connection load balancing.

① See Also

["About Run-Time Connection Load Balancing"](#)

- Create a connection affinity callback.
- Register the callback.

① Note

Transaction-based affinity is strictly scoped between the application/middle-tier and UCP. Therefore, transaction-based affinity requires only the `setFastConnectionFailoverEnabled` property be set to true and does not require complete FCF configuration.

In addition, transaction-based affinity does not technically require run-time connection load balancing. However, it can help with performance and is usually enabled regardless. If run-time connection load balancing is not enabled, the connection pool randomly picks connections.

This section contains the following subsections:

- [Creating a Connection Affinity Callback](#)
- [Registering a Connection Affinity Callback](#)
- [Removing a Connection Affinity Callback](#)

9.4.2.1 Creating a Connection Affinity Callback

Connection affinity requires the use of a callback. The callback is an implementation of the `ConnectionAffinityCallback` interface which is located in the `oracle.ucp` package. The callback is used by the connection pool to establish and retrieve a connection affinity context and is also used to set the affinity policy type (transaction-based or Web session).

The following example demonstrates setting an affinity policy in a callback implementation. The example also demonstrates manually setting an affinity context. typically, the connection pool sets the affinity context inside an application. However, the ability to manually set an affinity

context is provided for applications that want to customize affinity behavior and control the affinity context directly.

```
public class AffinityCallbackSample
    implements ConnectionAffinityCallback {

    Object appAffinityContext = null;
    ConnectionAffinityCallback.AffinityPolicy affinityPolicy =
        ConnectionAffinityCallback.AffinityPolicy.TRANSACTION_BASED_AFFINITY;

    //For Web session affinity, use WEBSSESSION_BASED_AFFINITY;

    public void setAffinityPolicy(AffinityPolicy policy)
    {
        affinityPolicy = policy;
    }

    public AffinityPolicy getAffinityPolicy()
    {
        return affinityPolicy;
    }

    public boolean setConnectionAffinityContext(Object affCxt)
    {
        synchronized (lockObj)
        {
            appAffinityContext = affCxt;
        }
        return true;
    }

    public Object getConnectionAffinityContext()
    {
        synchronized (lockObj)
        {
            return appAffinityContext;
        }
    }
}
```

9.4.2.2 Registering a Connection Affinity Callback

A connection affinity callback is registered on a connection pool using the `registerConnectionAffinityCallback` method. The callback is registered when creating the connection pool. Only one callback can be registered per connection pool.

The following example demonstrates registering a connection affinity callback implementation:

```
ConnectionAffinityCallback callback = new MyCallback();

PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionPoolName("AffinitySamplePool");
pds.registerConnectionAffinityCallback(callback);
...
```

9.4.2.3 Removing a Connection Affinity Callback

A connection affinity callback is removed from a connection pool using the `removeConnectionAffinityCallback` method. For example:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();

pds.setConnectionPoolName("AffinitySamplePool");
pds.removeConnectionAffinityCallback();
...
```

9.4.2.4 Strict Affinity Mode

By default, affinity is only a hint. A connection pool selects a new Oracle RAC instance for connections if it does not find a connection on a desired instance. You can change this behavior by switching the strict affinity mode on. The strict affinity mode throws a UCP exception if a connection on a desired instance is not found.

Use the following pool properties to switch on the strict affinity mode:

- The `useStrictWebSessionAffinity` property
Set the `useStrictWebSessionAffinity` property to `true` or `false` for switching the strict Web session affinity mode on or off respectively.
- The `useStrictXAAffinity` property
Set the `useStrictXAAffinity` property to `true` or `false` for switching the strict transaction-based affinity mode on or off respectively.

These properties can be handled through the `UniversalConnectionPoolMBean`.

Related Topics

- [UniversalConnectionPoolMBean](#)

9.5 Global Data Services

This section describes the new Global Data Services (GDS) feature that can be used with Universal Connection Pool:

- [Overview of Global Data Services](#)
- [Configuring an Application for Using GDS](#)

9.5.1 Overview of Global Data Services

Global Data Services (GDS) feature is available since Oracle Database 12c Release 1 (12.1). Through this feature, Fast Connection Failover, Run-time Connection Load-Balancing, and Connection Affinity features that were available only in Oracle RAC, were extended to a set of replicated databases offering common services.

The set of databases may include Oracle RAC and single-instance Oracle databases interconnected through Data Guard, GoldenGate, or any other replication technology. A database service that can be provided by multiple databases is called a global service, so that it can be distinguished from the traditional service that can be provided only by a single database. This combination enables services to be deployed anywhere within this globally distributed configuration, supporting load balancing, high availability, database affinity, and so on.

① See Also

Oracle Database Global Data Services Concepts and Administration Guide

9.5.2 Configuring an Application for Using GDS

UCP connects to Global Data Services in the same way that it connects to local services on an Oracle RAC. The service name in the connection string should be the name of the global service. The endpoint should be the endpoint of a GDS listener instead of the endpoint for the local, remote, or SCAN listener of a database.

A client must specify its region in the `REGION` parameter of the connection string. This is a new requirement for GDS. The region name is required because, in case of GDS, Run-time Load Balancing advisory is customized for particular regions. Following is an example of a typical connection string:

```
(DESCRIPTION=
  (ADDRESS=(GDS_protocol_address_information))
  (CONNECT_DATA=
    (SERVICE_NAME=global_service_name)
    (REGION=region_name)))
```

Like with local services, UCP can specify multiple GDS listeners in the same connection string for listener failover, load balancing, or both.

① Note

SCAN is not supported for GDS listeners, therefore endpoint for each listener must be specified.

```
(DESCRIPTION=
  (ADDRESS_LIST=
    (LOAD_BALANCE=ON)
    (FAILOVER=ON)
    (ADDRESS=(GDS_protocol_address_information))
    (ADDRESS=(GDS_protocol_address_information)))
  (CONNECT_DATA=
    (SERVICE_NAME=global_service_name)
    (REGION=region_name)))
```

The `REGION` parameter is optional if only global service managers from the local region are specified in the client connection string. This is the case when there is only one region in the GDS configuration, or can be the case when there are multiple regions. But, it is not feasible to change the connection string of the an existing client designed to work with a single database. If the `REGION` parameter is not specified, then the client's region is assumed to be the region of the global service manager used to connect to the global service.

① Note

Unless the `REGION` parameter is specified in the connection string, you can use a pre-12c thin JDBC client only with a GDS configuration that has a single region.

All GDS listeners in the preceding example belong to the same region where UCP is running, that is the local region. To provide high availability, when all GDSs in the local region are unavailable, you can specify the GDS listeners for the buddy region in additional ADDRESS_LIST descriptors.

```
(DESCRIPTION=
  (FAILOVER=on)
  (ADDRESS_LIST=
    (LOAD_BALANCE=ON)
    (ADDRESS=(global_protocol_address_information))
    (ADDRESS=(global_protocol_address_information)))
  (ADDRESS_LIST=
    (LOAD_BALANCE=ON)
    (ADDRESS=(global_protocol_address_information))
    (ADDRESS=(global_protocol_address_information)))
  (CONNECT_DATA=
    (SERVICE_NAME=global_service_name)
    (REGION=region_name)))
```

You do not need manual ONS configuration because UCP automatically retrieves the ONS connection information that is optimally customized for the UCP region from GDS.

Note

- To enable automatic ONS configuration for GDS, you must enable Fast Connection Failover (FCF) on UCP.
- Automatic ONS configuration works only with Oracle GDS and Oracle RAC. It does not work with single-instance Oracle Databases.

Automatic ONS configuration does not support ONS wallet or keystore parameters. If your application requires any of these parameters, then you must configure ONS explicitly in either of the following two ways:

- Calling the `PoolDataSource.setONSConfiguration(String)` method
- Adding the ONS wallet or keystore parameters in the local ONS configuration file

10

Ensuring Application Continuity

This chapter discusses the following concepts related to the Application Continuity feature of Oracle Database:

- [Overview of Ensuring Application Continuity with UCP](#)
- [Configuring the Data Source for Application Continuity](#)
- [Using Connection Labeling for Application Continuity](#)
- [Using Connection Initialization Callback for Application Continuity](#)

10.1 Overview of Ensuring Application Continuity with UCP

Oracle Database 12c Release 1 (12.1) introduced the Application Continuity feature that provides a general purpose, application-independent infrastructure. Application Continuity enables recovery of work from an application perspective, after the occurrence of a planned or unplanned outage that can be related to system, communication, or hardware following a repair, a configuration change, or a patch application.

For using Application Continuity, you must first configure your data source. After that, use one of the following two features for implementing Application Continuity in your applications using Universal Connection Pool (UCP):

- Using Connection Labeling for Application Continuity
- Using Connection Initialization Callback for Application Continuity

Related Topics

- [Configuring the Data Source for Application Continuity](#)
- [Using Connection Labeling for Application Continuity](#)
- [Using Connection Initialization Callback for Application Continuity](#)

10.2 Configuring the Data Source for Application Continuity

To utilize the Application Continuity feature on a pool-enabled data source, the application must make the following call on `oracle.ucp.jdbc.PoolDataSource` interface:

```
// pds is a PoolDataSource  
pds.setConnectionFactoryClassName("oracle.jdbc.replay.OracleDataSourceImpl");
```

Always connect to a service, instead of using SID. Application Continuity is not supported when connecting in the SID syntax.

When running against Oracle Real Application Clusters (Oracle RAC) or Data Guard, the application should also enable Fast Connection Failover (FCF) as shown in the following code snippet:

```
pds.setFastConnectionFailoverEnabled(true);
```

10.3 Using Connection Labeling for Application Continuity

Connection labeling enables an application to attach arbitrary name/value pairs to a connection. The application can request a connection with the desired label from the connection pool.

Connection labeling sets the initial state for each connection request. If the application uses connection labeling or benefits from labeling connections, then a labeling callback should be registered for Application Continuity to initialize clean connections at failover.

Every time Application Continuity gets a new connection from the underlying data source, the labeling callback executes. The callback executes during normal connection check-out and also during replay. So, the state that is created at run time is exactly re-created during replay. The initialization must be idempotent.

It is legal for the callback to execute a transaction as long as the transaction completes (either it commits or rolls back) at the end of callback invocation. Application Continuity repeats any action coded within the callback implementation, including such transaction. If an outage occurs during the execution of a UCP labeling callback, then Application Continuity may execute the callback more than once as part of the replay attempt. Again, it is important for the callback actions to be idempotent.

Related Topics

- [Labeling Connections in UCP](#)

10.4 Using Connection Initialization Callback for Application Continuity

If an application cannot use connection labeling because it cannot be changed, then the connection initialization callback is provided for such an application.

When registered, the initialization callback is executed every time a connection is borrowed from the pool and at each successful reconnection following a recoverable error..

Related Topics

- [About Connection Initialization Callback](#)

Shared Pool for Sharded Databases

This chapter describes UCP Shared Pool for sharded database in the following sections:

- [Overview of UCP Shared Pool for Database Sharding](#)
- [About Handling Connection Requests for a Sharded Database](#)
- [Middle-Tier Routing Using UCP](#)
- [UCP APIs for Database Sharding Support](#)
- [UCP APIs for Middle-Tier Routing Support](#)
- [UCP Sharding Example](#)
- [Middle-Tier Routing with UCP Example](#)

11.1 Overview of UCP Shared Pool for Database Sharding

Starting from Oracle Database 12c Release 2 (12.2.0.1), Universal Connection Pool (UCP) supports database sharding. UCP recognizes the sharding keys specified and connects to the specific shard. Sharding uses Global Data Services (GDS), where GDS routes a client request to an appropriate database, based on various parameters such as availability, load, network latency, and replication lag.

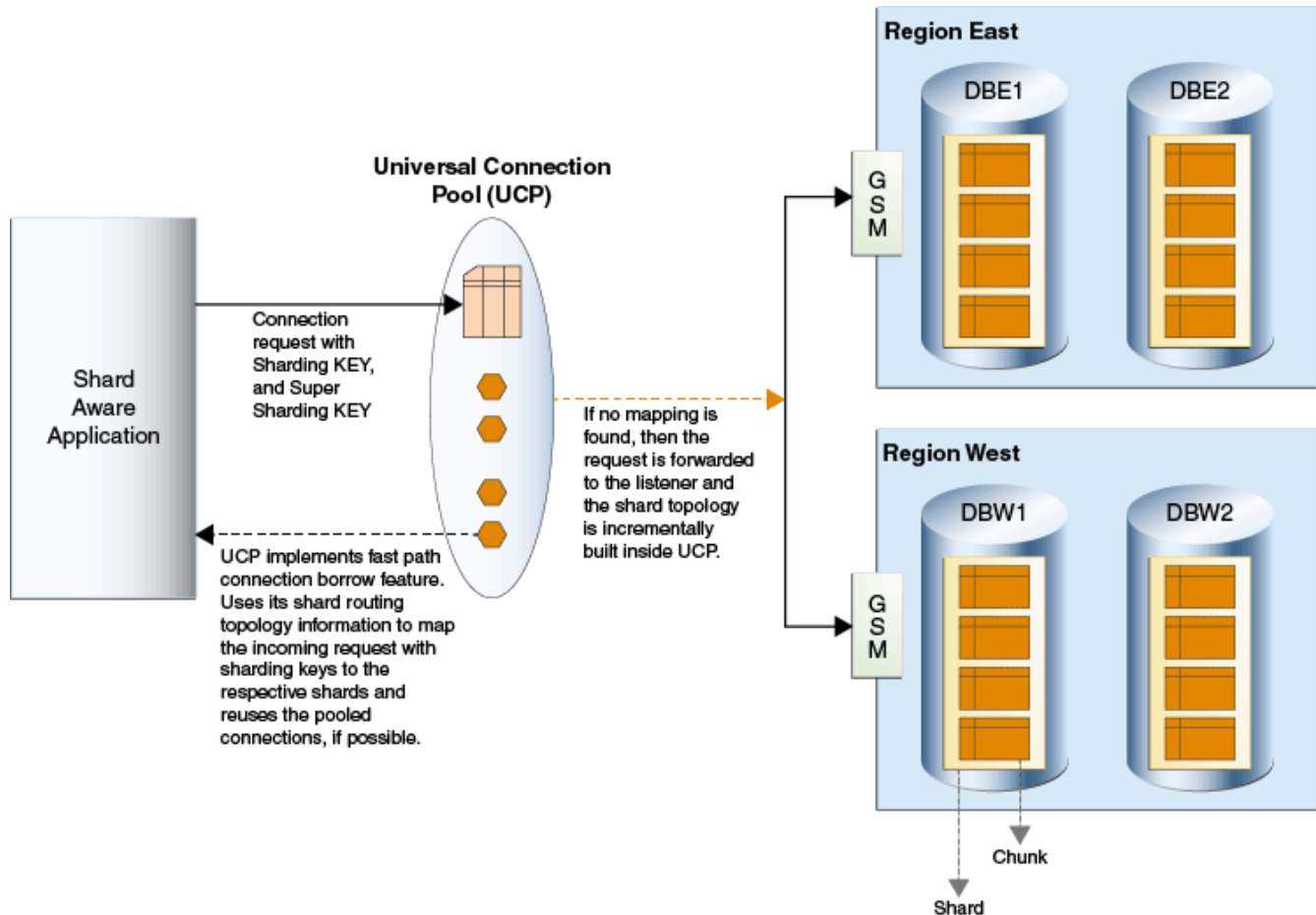
① See Also

- *Oracle Database JDBC Developer's Guide*
- *Oracle Database Administrator's Guide*

Use Case of UCP Shared Pool for Database Sharding

This section describes a use case of UCP Shared Pool for database sharding. In the use case, the applications connecting to sharded database use UCP to store connections to different shards and chunks of the sharded GDS database within the same Shared Pool. The applications must provide the sharding key to UCP during the connection request. Based on the sharding key, the pool routes the connection request to the correct shard. The data distribution across the shards and chunks in the database is transparent to the user. UCP transparently handles resharding and chunk movements, minimizing the impact on the end users.

The following diagram illustrates this use case:

Figure 11-1 Universal Connection Pool (UCP) Using Sharded Database Architecture**Related Topics**

- [Global Data Services](#)

11.2 About Handling Connection Requests for a Sharded Database

The following sections describe how Universal Connection Pool (UCP) handles connection requests for a sharded database:

- [About Building the Sharding Key](#)
- [How to Checkout Connections from a Pool with a Known Sharding key](#)
- [About Checking Out Connections without Providing the Sharding Keys](#)
- [About Connecting to the Catalog Database for Cross Shard Queries](#)
- [About Configuring the Number of Connections Per Shard](#)
- [Pool Connection Selection Algorithm During Connection Checkout](#)
- [Failover or Resharding Event handling in UCP](#)

11.2.1 About Building the Sharding Key

The shard aware applications must identify and build the sharding key and the super sharding key, which are required to establish a connection to the sharded database. For achieving this, the shard aware applications must use the `OracleShardingKey` and the `OracleShardingKeyBuilder` interfaces.

The `OracleShardingKeyBuilder` uses the following builder method for supporting compound keys with different data types:

```
subkey(Object subkey, java.sql.SQLTYPE subkeyDataType)
```

There are multiple invocations of the `subkey` method on the builder for building a compound sharding key, where each subkey can be of different data types. The data type can be defined using the `oracle.jdbc.OracleType` enum or `java.sql.JDBCType`.

Example 11-1 Building a Sharding Key

The following example shows how to build a sharding key:

```
import java.sql.Connection;
import java.sql.Date;
import java.sql.SQLException;
import java.sql.Statement;

import oracle.jdbc.OracleShardingKey;
import oracle.jdbc.OracleType;
import oracle.ucp.jdbc.PoolDataSource;
import oracle.ucp.jdbc.PoolDataSourceFactory;

public class ShardExample
{
    public static void main(String[] args) throws SQLException
    {
        String url = "jdbc:oracle:thin:@(DESCRIPTION=(ADDRESS=(HOST=myhost)(PORT=3216)
(PROTOCOL=tcp))(CONNECT_DATA=(SERVICE_NAME=myservice)(REGION=east)))";
        String user="testuser1";
        String pwd = "password";

        PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
        pds.setURL(url);
        pds.setUser(user);
        pds.setPassword(pwd);
        pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
        pds.setInitialPoolSize(5);
        pds.setMinPoolSize(5);
        pds.setMaxPoolSize(20);

        // build the sharding key object
        Date shardingKeyVal = new java.sql.Date(0L);
        OracleShardingKey sdkey = pds.createShardingKeyBuilder()
            .subkey(shardingKeyVal, OracleType.DATE)
            .build();
    }
}
```

```
Connection conn = pds.createConnectionBuilder()
    .shardingKey(sdkey)
    .build();

Statement stmt = conn.createStatement();
stmt.execute("... SQL statement here ...");
stmt.close();
conn.close();
}
}
```

The following code snippet shows how to build a compound sharding key that consists of String and Date data types:

```
...
Date shardingKeyVal = new java.sql.Date(0L);
...
OracleShardingKey shardingKey = datasource.createShardingKeyBuilder()
    .subkey("abc@xyz.com", JDBCType.VARCHAR)
    .subkey(shardingKeyVal, OracleType.DATE)
    .build();
...
```

Note

- There is a fixed set of data types that are valid and supported. If any unsupported data types are used as keys, then exceptions are thrown. The following list specifies the supported data types:
 - OracleType.VARCHAR2/JDBCType.VARCHAR
 - OracleType.CHAR/JDBCType.CHAR
 - OracleType.NVARCHAR/JDBCType.NVARCHAR
 - OracleType.NCHAR/JDBCType.NCHAR
 - OracleType.NUMBER/JDBCType.NUMERIC
 - OracleType.FLOAT/JDBCType.FLOAT
 - OracleType.DATE/JDBCType.DATE
 - OracleType.TIMESTAMP/JDBCType.TIMESTAMP
 - OracleType.TIMESTAMP_WITH_LOCAL_TIME_ZONE
 - OracleType.RAW
- You must provide a sharding key that is compliant to the NLS formatting specified in the database.

11.2.2 How to Checkout Connections from a Pool with a Sharding Key

When a connection is borrowed from UCP, then the shard aware application can provide the sharding key and the super sharding key using the new connection builder present in the PoolDataSource class. If sharding keys do not exist or do not map to the data types specified by

the database metadata, then an `IllegalArgumentException` is thrown. The following code snippet shows how to checkout a connection with sharding keys:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
...
Connection conn = pds.createConnectionBuilder()
// Establish a connection using sharding key and super sharding key
    .shardingKey(shardingKey)
    .superShardingKey(superShardingKey)
    .build();

OracleShardingKey shardKey = pds.createShardingKeyBuilder()
// Build a compound sharding key with email address and customer ID as the two sharding keys
    .subkey(<email>, OracleType.VARCHAR2)
    .subkey(<custid>, OracleType.NUMBER)
    .build();

OracleShardingKey superShardKey = pds.createShardingKeyBuilder()
// Build a super sharding key with the customer region
    .subkey(<cust_region>, OracleType.VARCHAR2)
    .build();
```

Note

You must specify a sharding key during the connection checkout. Otherwise, an error or exception is thrown back to the application. Race condition also results in exception during connection usage.

11.2.3 About Checking Out Connections without Providing the Sharding Keys

Providing sharding keys in a connection request through connection builder API is mandatory when you use UCP data source for connecting to a sharded database. If you do not provide the sharding key, then an exception is thrown back to the user.

11.2.4 About Connecting to the Shard Catalog or Co-ordinator for Multi Shard Queries

When connecting to the Shard Catalog or Co-ordinator for running multi shard queries, it is recommended that a separate pool be created using a new `PoolDataSource` instance. You can run multi shard queries on connections retrieved from a data source that is created on the coordinator service. The connection request for the coordinator should not have sharding keys in the connection builder API.

11.2.5 About Configuring the Number of Connections Per Shard

When UCP is used to pool connections for a sharded database, the pool contains connections to different shards. So, when connections are pulled, to ensure a fair usage of the pool

capacity across all shards connected, UCP uses the `MaxConnectionsPerShard` parameter. This is a global parameter, which applies to every shard in the sharded database, and is used to limit the total number of connections to any shard below the specified limit.

The following table describes the APIs for setting and retrieving this parameter:

Method	Description
<code>poolDataSource.setMaxConnectionsPerShard(<max_connections_per_shard_limit>)</code>	Sets the maximum number of connections per shard.
<code>poolDataSource.getMaxConnectionsPerShard()</code>	Retrieves the value that was set using the <code>setMaxConnectionsPerShard(<max_connections_per_shard_limit>)</code> method.

Note

You cannot use the `MaxConnectionsPerShard` parameter in a sharded database with Oracle Golden Gate configuration.

11.2.6 Pool Connection Selection Algorithm During Connection Checkout

Whenever new connections are created through UCP to different shards in the sharded database, the pool incrementally learns and builds a shard routing cache internally.

The routing cache maps the sharding keys to the respective shards, on which the keys exist. While looking up connections in the pool for a connection request with specific sharding keys, UCP uses the cache to redirect the request to the correct shard. This feature, called Fast Path Connection Borrow, enables efficient reuse of connections in the pool, based on the requested sharding keys. This feature also helps in avoiding going to the sharded database for routing the requests.

11.2.7 Failover or Resharding Event Handling in UCP

After a resharding or failover event, an attempt is made to keep the UCP shard routing cache in sync with the data on the server. The cache is kept up-to-date by subscribing to the ONS notification for various changes on the database.

11.3 Middle-Tier Routing Using UCP

Starting from Oracle Database Release 18c, Oracle Universal Connection Pool (UCP) introduces the Middle-Tier Routing feature. This feature helps the Oracle customers, who use the Sharding feature, to have a dedicated middle tier from the client applications to the sharded database.

Typically, the middle-tier connection pools route database requests to specific shards. During such a routing, each middle-tier connection pool establishes connections to each shard, creating too many connections to the database. The Middle-Tier Routing feature solves this problem by having a dedicated middle tier (Web Server or Application Server) for each Data Center or Cloud, and routing client requests directly to the relevant middle tier, where the shard containing the client data (corresponding to the client sharding key) resides.

The `OracleShardRoutingCache` class in UCP provides middle-tier routing APIs that can be used to route the client requests to the appropriate middle tier. An instance of this class represents the internal shard routing cache of UCP, which can be created by providing connection properties such as user, password, and URL of the sharding catalog. Starting from Oracle Database Release 19c, you must also specify a new connection property, `serviceName`. It is the name of the global service name.

The routing cache connects to the sharding catalog to retrieve the key to shard mapping topology and stores it in its cache. The `getShardInfoForKey(shardKey, superShardKey)` method uses the routing cache of UCP to get the information about shards for the specified sharding key. The `ShardInfo` instance encapsulates a unique shard name and priority of the shard. The application using the middle-tier API can map the returned unique shard name value to a middle tier that has connections to a specific shard.

The routing cache is automatically refreshed or updated on chunk move or split by subscribing to respective ONS events.

11.4 UCP APIs for Database Sharding Support

The `UCPConnectionBuilder` Class

The `UCPConnectionBuilder` class is used for building connection objects with additional parameters other than the username, password, and label. To use the builder, you must call the corresponding builder method for each parameter that needs to be a part of the connection request, followed by a `build` method. The order in which the builder methods are called is not important. However, if the same builder attribute is applied more than once, then only the most recent value is considered while building the connection.

Syntax

```
public abstract class UCPConnectionBuilder<S> implements
OracleConnectionBuilder<UCPConnectionBuilder<S>,S>
```

The `UCPConnectionBuilder` class also provides the `validate` method and several constructors for setting the data for a specific user.

Example 11-2 Creating the Connection Builder

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
//set the required properties on the datasource
ShardingKey superShardingKey = ds.createShardingKeyBuilder()
                                .subkey("EASTERN_REGION", JDBCType.VARCHAR)
                                .build();
ShardingKey superShardingKey = ds.createShardingKeyBuilder()
                                .subkey("PITTSBURGH_BRANCH", JDBCType.VARCHAR)
                                .build();
Connection conn = pds.createConnectionBuilder()
                    .shardingKey(superShardingKey)
                    .superShardingKey(superShardingKey)
                    .build();
```

New Methods in PoolDataSource Interface

The following methods have been introduced in the `oracle.ucp.jdbc.PoolDataSource` interface:

```

/**
 * Creates a new UCPCConnectionBuilder instance.
 *
 * @param <S>
 * Connection type for this ConnectionBuilder
 * @param <B>
 * Builder type to use
 * @return The OracleConnectionBuilder instance that was created
 */
public UCPCConnectionBuilder createConnectionBuilder();

/**
 * Creates a new OracleShardingKeyBuilder instance
 *
 * @return The OracleShardingKeyBuilder instance that was created
 */
public default OracleShardingKeyBuilder createShardingKeyBuilder() {
    return new OracleShardingKeyBuilderImpl();
}

```

New Method in PoolXADataSource Interface

The following method has been introduced in the `oracle.ucp.admin.UniversalConnectionPoolManager` interface:

```

/**
 * Creates a new XAConnectionBuilder instance.
 *
 * @return The XAConnectionBuilder instance that was created
 */
public UCPCXAConnectionBuilder createXAConnectionBuilder();

```

11.5 UCP APIs for Middle-Tier Routing Support

The OracleShardRoutingCache Class

This class extends the internal shard routing cache of UCP and makes the basic routing cache feature available to the WebLogic Server or middle-tier routers or load balancers.

```
public class OracleShardRoutingCache extends ShardRoutingCache
```

This class provides the `OracleShardRoutingCache(Properties dataSourceProps)` and `Set<ShardInfo> getShardInfoForKey(OracleShardingKey key, OracleShardingKey superKey)` methods.

The ShardInfo Interface

The `ShardInfo` interface instances encapsulate unique shard name and priority. The unique shard name can then be mapped to a middle-tier server that connects to a specific shard.

11.6 UCP Sharding Example

Example

The following code snippet shows how to use UCP sharding APIs:

Example 11-3 UCP Sharding Example

```
PoolDataSource pds = new PoolDataSourceImpl();
pds.setURL(url);
pds.setUser("system");
pds.setPassword("manager");
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");

OracleShardingKey employeeNamekey =
    pds.createShardingKeyBuilder()
        .subkey("Mary", JDBCType.VARCHAR) // First Name
        .subkey("Claire", JDBCType.VARCHAR) // Last Name
        .build();

OracleShardingKey locationKey = pds.createShardingKeyBuilder()
    .subkey("US", JDBCType.VARCHAR)//Location
    .build();

OracleConnection connection = pds.createConnectionBuilder()
    .shardingKey(employeeNamekey)
    .superShardingKey(locationKey)
    .build();
```

11.7 Middle-Tier Routing with UCP Example

The following example explains the usage of the middle-tier routing API of UCP.

Example 11-4 Example of Middle-Tier Routing Using UCP

```
import java.sql.SQLException;
import java.util.Properties;
import java.util.Random;
import java.util.Set;

import oracle.jdbc.OracleShardingKey;
import oracle.jdbc.OracleType;
import oracle.ucp.UniversalConnectionPoolException;
import oracle.ucp.routing.ShardInfo;
import oracle.ucp.routing.oracle.OracleShardRoutingCache;

/**
 * The code example illustrates the usage of the middle-tier routing feature of UCP.
 * The API accepts sharding key as input and returns the set of ShardInfo
 * instances mapped to the sharding key. The ShardInfo instance encapsulates
 * unique shard name and priority. The unique shard name then can be mapped
 * to a middle-tier server that connects to a specific shard.
 */
```

```
*/
public class MidtierShardingExample {

    private static String user = "testuser1";
    private static String password = "testuser1";

    // catalog DB URL
    private static String url = "jdbc:oracle:thin:@//hostName:1521/catalogServiceName";
    private static String region = "regionName";

    public static void main(String args[]) throws Exception {
        testMidTierRouting();
    }

    static void testMidTierRouting() throws UniversalConnectionPoolException,
        SQLException {

        Properties dbConnectProperties = new Properties();
        dbConnectProperties.setProperty(OracleShardRoutingCache.USER, user);
        dbConnectProperties.setProperty(OracleShardRoutingCache.PASSWORD, password);
        // Mid-tier routing API accepts catalog DB URL
        dbConnectProperties.setProperty(OracleShardRoutingCache.URL, url);

        // Region name is required to get the ONS config string
        dbConnectProperties.setProperty(OracleShardRoutingCache.REGION, region);

        OracleShardRoutingCache routingCache = new OracleShardRoutingCache(
            dbConnectProperties);

        final int COUNT = 10;
        Random random = new Random();

        for (int i = 0; i < COUNT; i++) {
            int key = random.nextInt();
            OracleShardingKey shardKey = routingCache.getShardingKeyBuilder()
                .subkey(key, OracleType.NUMBER).build();
            OracleShardingKey superShardKey = null;

            Set<ShardInfo> shardInfoSet = routingCache.getShardInfoForKey(shardKey,
                superShardKey);

            for (ShardInfo shardInfo : shardInfoSet) {
                System.out.println("Sharding Key=" + key + " Shard Name="
                    + shardInfo.getName() + " Priority=" + shardInfo.getPriority());
            }
        }
    }
}
```

12

Diagnosing a Connection Pool

The following parameters are used for diagnosing Universal Connection Pool (UCP):

- [Pool Statistics](#)
- [Dynamic Monitoring Service Metrics](#)
- [About Viewing Oracle RAC Statistics](#)
- [Overview of Logging in UCP](#)
- [Exceptions and Error Codes](#)

12.1 Pool Statistics

Universal Connection Pool (UCP) provides a set of run-time statistics for the connection pool. These statistics can be divided into the following two categories:

- **Noncumulative**
These statistics apply only to the current running connection pool instance.
- **Cumulative**
These statistics are collected across multiple pool start/stop cycles.

The `oracle.ucp.UniversalConnectionPoolStatistics` interface provides methods that are used to query the connection pool statistics. The methods of this interface can be called from a pool-enabled data source and pool-enabled XA data source, using the `oracle.ucp.jdbc.PoolDataSource.getStatistics` method. For example:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
...
...
int totalConnsCount = pds.getStatistics().getTotalConnectionsCount();
System.out.println("The total connection count in the pool is "+ totalConnsCount +".");
```

The `oracle.ucp.jdbc.PoolDataSource.getStatistics` method can also be called by itself to return all connection pool statistics as a String.

12.2 Dynamic Monitoring Service Metrics

UCP supports all the pool statistics to be in the form of Dynamic Monitoring Service (DMS) metrics. You must include the `dms.jar` file in the class path of the application to collect and utilize these DMS metrics.

UCP supports DMS metrics collection in both the pool manager interface and the pool manager MBean. You can use the `UniversalConnectionPoolManager.startMetricsCollection` method to start collecting DMS metrics for the specified connection pool instance, and use the `UniversalConnectionPoolManager.stopMetricsCollection` method to stop DMS metrics collection. The metrics update interval can be specified using the `UniversalConnectionPoolManager.setMetricUpdateInterval` method. The pool manager MBean exports similar operations.

12.3 About Viewing Oracle RAC Statistics

UCP provides a set of Oracle RAC run-time statistics that are used to determine how well a connection pool is utilizing Oracle RAC features and are also used to help determine whether the connection pool has been configured properly to use the Oracle RAC features. The statistics report FCF processing information, run-time connection load balance success/failure rate, and affinity context success/failure rate.

The `OracleJDBCConnectionPoolStatistics` interface that is located in the `oracle.ucp.jdbc.oracle` package provides methods that are used to query the connection pool for Oracle RAC statistics. The methods of this interface can be called from a pool-enabled and pool-enabled XA data source using the data source's `getStatistics` method. For example:

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
...
```

```
Long rclbS = ((OracleJDBCConnectionPoolStatistics)pds.getStatistics()).
    getSuccessfulRCLBBasedBorrowCount();
System.out.println("The RCLB success rate is "+rclbS+".");
```

The data source's `getStatistics` method can also be called by itself and returns all connection pool statistics as a `String` and includes the Oracle RAC statistics.

12.3.1 Fast Connection Failover Statistics

The `getFCFProcessingInfo` method provides information on recent Fast Connection Failover (FCF) attempts in the form of a `String`. The FCF information is typically used to help diagnose FCF problems. The information includes the outcome of each FCF attempt (successful or failed), the relevant Oracle RAC instances, the number of connections that were cleaned up, the exception that triggered the FCF attempt failure, and more. The following example demonstrates using the `getFCFProcessingInfo` method:

```
String fcfInfo = ((OracleJDBCConnectionPoolStatistics)pds.getStatistics()).
    getFCFProcessingInfo();
System.out.println("The FCF information: "+fcfInfo+".");
```

Following is a sample output string from the `getFCFProcessingInfo()` method:

```
Oct 28, 2008 12:34:02 SUCCESS <Reason:planned> <Type:SERVICE_UP> \
<Service:"svvc1"> <Instance:"inst1"> <Db:"db1"> \
Connections:(Available=6 Affected=2 FailedToProcess=0 MarkedDown=2 Closed=2) \
(Borrowed=6 Affected=2 FailedToProcess=0 MarkedDown=2 MarkedDeferredClose=0 Closed=2) \
TornDown=2 MarkedToClose=2 Cardinality=2
...
Oct 28, 2008 12:09:52 SUCCESS <Reason:unplanned> <Type:SERVICE_DOWN> \
<Service:"svc1"> <Instance:"inst1"> <Db:"db1"> \
Connections:(Available=6 Affected=2 FailedToProcess=0 MarkedDown=2 Closed=2) \
(Borrowed=6 Affected=2 FailedToProcess=0 MarkedDown=2 MarkedDeferredClose=0 Closed=2)
...
Oct 28, 2008 11:14:53 FAILURE <Type:HOST_DOWN> <Host:"host1"> \
Connections:(Available=6 Affected=4 FailedToProcess=0 MarkedDown=4 Closed=4) \
(Borrowed=6 Affected=4 FailedToProcess=0 MarkedDown=4 MarkedDeferredClose=0 Closed=4)
```

If you enable logging, then the preceding information will also be available in the UCP logs and you will be able to verify the FCF outcome.

12.3.2 Run-Time Connection Load Balance Statistics

The run-time connection load balance statistics are used to determine if a connection pool is effectively utilizing the run-time connection load balancing feature of Oracle RAC. The statistics report how many requests successfully used the run-time connection load balancing algorithms and how many requests failed to use the algorithms. The `getSuccessfulRCLBBasedBorrowCount` method and the `getFailedRCLBBasedBorrowCount` method, respectively, are used to get the statistics. The following example demonstrates using the `getFailedRCLBBasedBorrowCount` method:

```
Long rclbF = ((OracleJDBCCConnectionPoolStatistics)pds.getStatistics()).  
    getFailedRCLBBasedBorrowCount();  
System.out.println("The RCLB failure rate is: "+rclbF+"");
```

A high failure rate may indicate that the Oracle RAC Load Balancing Advisory or connection pool is not configured properly.

12.3.3 Connection Affinity Statistics

The connection affinity statistics are used to determine if a connection pools is effectively utilizing connection affinity. The statistics report the number of borrow requests that succeeded in matching the affinity context and how many requests failed to match the affinity context. The `getSuccessfulAffinityBasedBorrowCount` method and the `getFailedAffinityBasedBorrowCount` method, respectively, are used to get the statistics. The following example demonstrates using the `getFailedAffinityBasedBorrowCount` method:

```
Long affF = ((OracleJDBCCConnectionPoolStatistics)pds.getStatistics()).  
    getFailedAffinityBasedBorrowCount();  
System.out.println("The connection affinity failure rate is: "+affF+"");
```

12.4 Overview of Logging in UCP

UCP leverages the JDK logging facility (`java.util.logging`). Logging is not enabled by default and must be configured in order to print log messages. Logging can be configured using a log configuration file as well as through API-level configuration.

Note

The default log level is `null`. This ensures that a parent logger's log level is used by default.

12.4.1 Using a Logging Properties File

Logging can be configured using a properties file. The location of the properties file must be set as a Java property for the logging configuration file property. For example:

```
java -Djava.util.logging.config.file=log.properties
```

The logging properties file defines the handler to use for writing logs, the formatter to use for formatting logs, a default log level, as well as log levels for specific packages or classes. For example:

```
handlers = java.util.logging.ConsoleHandler  
java.util.logging.ConsoleHandler.level = ALL
```

```
java.util.logging.ConsoleHandler.formatter = java.util.logging.SimpleFormatter
```

```
oracle.ucp.level = FINEST  
oracle.ucp.jdbc.PoolDataSource = WARNING
```

A custom formatter is included with UCP and can be entered as the value for the formatter property. For example:

```
java.util.logging.ConsoleHandler.formatter = oracle.ucp.util.logging.UCPFormatter
```

You can also download the `ucpdemos.jar` file, which is shipped with UCP, from Oracle Technology Network (OTN). This file contains a list of sample logging property files. For example, this file contains the logging property file that can be used for troubleshooting the Fast Connection Failover (FCF) feature.

12.4.2 Using UCP and JDK API

Logging can be dynamically configured through either the UCP API or the JDK API. When using the UCP API, logging is configured using a connection pool manager. When using the JDK, logging is configured using the `java.util.logging` implementation.

The following example demonstrates using the UCP API to configure logging:

```
UniversalConnectionPoolManager mgr = UniversalConnectionPoolManagerImpl.  
getUniversalConnectionPoolManager();  
  
mgr.setLogLevel(Level.FINE);
```

The following example demonstrate using the JDK logging implementation directly:

```
Logger.getLogger("oracle.ucp").setLevel(Level.FINEST);  
Logger.getLogger("oracle.ucp.jdbc.PoolDataSource").setLevel(Level.FINEST);
```

12.4.3 Enabling or Disabling Feature-Specific Logging at Runtime

Starting from Oracle Database 12c Release 2 (12.2.0.1), UCP provides support for enabling and disabling logging for selected features during runtime. For example, you can enable logging only for Load Balancing feature, while disabling logging for other features of UCP. Again, during the same run, you can enable logging for Fast Failover feature and disable logging for Load Balancing feature.

By default, logging for all features is enabled.

The logging switching feature of UCP is a part of the `OracleDiagnosabilityMBean`. For using this bean, start JConsole and connect to the application.

Displaying Supported Features

For a list of supported features, use the following method:

```
getTraceController().getSupportedFeatures()
```

Displaying Enabled Features

For a list of currently enabled features, use the following method:

```
getTraceController().getEnabledFeatures()
```

Enabling Logging for a Feature

For enabling logging for a specific feature or for all features, use the `trace` method in the following ways:

```
trace(boolean enable, String feature_name)
trace(boolean enable, ALL)
```

Disabling Logging for a Feature

For disabling logging for a specific feature or for all features, use the `trace` method in the following ways:

```
trace(boolean disable, String feature_name)
trace(boolean disable, ALL)
```

Suspending and Resuming Logging

Use the following methods for suspending and resuming logging respectively:

```
suspend()
resume()
```

12.4.4 About Using the Logging Properties File for Feature-Specific Logging

Starting from Oracle Database 12c Release 2 (12.2.0.1), you can enable or disable logging for specific features by adding a property in the logging properties file. By default, logging is enabled for all features. Otherwise, you can enable logging for all features using the following syntax: `clio.feature.all = on`. For feature-specific enabling of logging, you can use the properties as mentioned in the following section:

Supported Features for feature-Based Granularity

```
clio.feature.pool_statistics      = on
clio.feature.check_in            = on
clio.feature.check_out           = on
clio.feature.labeling            = on
clio.feature.conn_construction   = on
clio.feature.conn_destruction    = on
clio.feature.high_availability   = on
clio.feature.load_balancing      = on
clio.feature.transaction_affinity = on
clio.feature.web_affinity        = on
clio.feature.data_affinity       = on
clio.feature.conn_harvesting     = on
clio.feature.ttl_conn_timeout    = on
clio.feature.abandoned_conn_timeout = on
clio.feature.admin               = on
clio.feature.sharding            = on
```

12.4.5 Supported Log Levels

The following list describes each of the log levels that are supported for JDBC. Levels lower than `FINE` produce output that may not be meaningful to users. Levels lower than `FINER` will produce very large volumes of output.

- INTERNAL_ERROR – Internal Errors
- SEVERE – SQL Exceptions
- WARNING – SQL Warnings and other invisible problems
- INFO – Public events such as connection attempts or Oracle RAC events
- CONFIG – SQL statements
- FINE – Public APIs
- TRACE_10 – Internal events
- FINER – Internal APIs
- TRACE_20 – Internal debug
- TRACE_30 – High volume internal APIs
- FINEST – High volume internal debug

12.5 Exceptions and Error Codes

Many UCP methods throw the `UniversalConnectionPoolException`, with exception chaining supported. You can call the `printStackTrace` method on the thrown exception, to identify the root cause of the exception. The `UniversalConnectionPoolException` includes standard Oracle error codes that are in the range of 45000 and 45499. The `getErrorCode` method can be used to retrieve the error code for an exception.

A

Error Codes Reference

This appendix briefly discusses the general structure of Universal Connection Pool (UCP) error messages, UCP error messages for the connection pool layer, and UCP error messages for JDBC data sources and dynamic proxies. The appendix is organized as follows:

- [General Structure of UCP Error Messages](#)
- [Connection Pool Layer Error Messages](#)
- [JDBC Data Sources and Dynamic Proxies Error Messages](#)

Both the message lists are sorted by the error message number.

A.1 General Structure of UCP Error Messages

The general UCP error message structure enables run-time information to be appended to the end of a message, following a colon, as follows:

<error_message>:<extra_info>

For example, a closed statement error might be displayed as follows:

Closed Statement:next

This indicates that the exception was thrown during a call to the `next` method (of a result set object).

In some cases, the user can find the same information in a stack trace.

A.2 Connection Pool Layer Error Messages

This section lists UCP error messages for the connection pool layer.

Table A-1 Connection Pool Layer Error Messages

Error Message Number	Message
UCP-45001	Universal Connection Pool internal error
UCP-45002	No available connections in the Universal Connection Pool
UCP-45003	Universal Connection Pool already exists
UCP-45004	Invalid connection retrieval information
UCP-45005	Callback already registered
UCP-45006	Invalid Universal Connection Pool configuration
UCP-45051	Inactive connection timeout timer scheduling failed
UCP-45052	Abandoned connection timeout timer scheduling failed
UCP-45053	Time-to-live connection timeout timer scheduling failed
UCP-45054	The Universal Connection Pool cannot be null

Table A-1 (Cont.) Connection Pool Layer Error Messages

Error Message Number	Message
UCP-45055	Error when removing an available connection
UCP-45057	The AvailableConnections object cannot be null
UCP-45058	The Failoverable object cannot be null
UCP-45059	MaxPoolsize is set to 0. There are no connections to return
UCP-45060	Invalid life cycle state. Check the status of the Universal Connection Pool
UCP-45061	Universal Connection Pool is not started. Start the Universal Connection Pool before accessing
UCP-45062	The collection of available connections can only be set when the Universal Connection Pool is in the initialization state
UCP-45063	Universal Connection Pool has been shutdown while attempting to get a connection
UCP-45064	All connections in the Universal Connection Pool are in use
UCP-45065	Connection borrowing returned null
UCP-45091	Connection labeling callback already registered
UCP-45092	Borrowing labeled connection with no labeling callback registered
UCP-45093	Requested no-label connection but borrowing labeled connection
UCP-45097	Connection harvesting timer scheduling failed
UCP-45100	ConnectionFactoryAdapter returned null
UCP-45103	ConnectionFactoryAdapter must be an instance of DataSourceConnectionFactoryAdapter
UCP-45104	ConnectionFactoryAdapter object cannot be null
UCP-45105	ConnectionFactoryAdapter must be an instance of ConnectionPoolDataSourceConnectionFactoryAdapter
UCP-45106	ConnectionFactoryAdapter must be an instance of XADataSourceConnectionFactoryAdapter
UCP-45150	UniversalPooledConnection cannot be null
UCP-45152	UniversalPooledConnectionStatus object cannot be null
UCP-45153	The connection label key cannot be null or an empty string
UCP-45154	The connection labeling operation cannot be invoked on closed connections
UCP-45155	Connection harvesting callback already registered
UCP-45156	Abandoned connection timeout callback already registered
UCP-45157	Time-to-live connection timeout callback already registered
UCP-45201	The connection label key cannot be null or an empty string
UCP-45202	The cloning of the ConnectionRetrievalInfo object failed
UCP-45203	The Connection Request Info is null
UCP-45251	ConnectionPoolDataSource cannot be null
UCP-45252	Invalid ConnectionRetrievalInfo object
UCP-45253	SQLException occurred while getting PooledConnection from ConnectionPoolDataSource
UCP-45254	Invalid connection type. Must be a javax.sql.PooledConnection

Table A-1 (Cont.) Connection Pool Layer Error Messages

Error Message Number	Message
UCP-45255	SQLException while closing PooledConnection
UCP-45256	Data source cannot be null
UCP-45257	Cannot get Connection from Data source
UCP-45258	Invalid connection type. Must be a java.sql.Connection
UCP-45259	The connection to proxy must be an instance of java.sql.Connection
UCP-45260	XADatasource cannot be null
UCP-45261	SQLException occurred while getting XAConnection from XADataSource
UCP-45262	Invalid connection type. Must be a javax.sql.XAConnection
UCP-45263	SQLException occurred while closing XAConnection
UCP-45264	The connection cannot be null
UCP-45265	The connection to proxy must be an instance of java.sql.Statement
UCP-45266	The statement to proxy must be an instance of java.sql.ResultSet
UCP-45267	The connection to proxy must be an instance of javax.sql.XAConnection
UCP-45268	The Driver argument cannot be null
UCP-45269	The URL argument cannot be null
UCP-45301	Unable to get a connection for failover information
UCP-45302	Unable to execute SQL query to get failover information
UCP-45303	SQLException occurred while getting failover information
UCP-45304	The event type cannot be null
UCP-45305	The event type is invalid. Event type must be database/event/host or database/event/service
UCP-45306	The failover event type is invalid. It must be an OracleFailoverEvent
UCP-45307	The affinity context is invalid. It must be an OracleConnectionAffinityContext
UCP-45308	Exception occurred while enabling failover with remote ONS subscription
UCP-45350	Universal Connection Pool already exists in the Universal Connection Pool Manager. Universal Connection Pool cannot be added to the Universal Connection Pool Manager
UCP-45351	Universal Connection Pool not found in Universal Connection Pool Manager. Register the Universal Connection Pool with Universal Connection Pool Manager
UCP-45352	Cannot get Universal Connection Pool Manager instance
UCP-45353	Cannot get Universal Connection Pool Manager MBean instance
UCP-45354	MBean ObjectName is not in the right format. Use the right format to construct ObjectName for MBean
UCP-45355	MBean exception occurred while registering or unregistering the MBean
UCP-45356	MBean already exists in the MBeanServer. Use a different name to register MBean
UCP-45357	Exception occurred when trying to register an object in the MBean server that is not a JMX compliant MBean
UCP-45358	The specified MBean does not exist in the repository
UCP-45359	Invalid target object type is specified. Check the managed resource

Table A-1 (Cont.) Connection Pool Layer Error Messages

Error Message Number	Message
UCP-45360	Invalid MBean Descriptor is specified. Check the Universal Connection Pool Manager MBean Descriptor
UCP-45361	Runtime exception occurred while building MBeanInfo for Universal Connection Pool Manager MBean
UCP-45362	Runtime exception occurred while building constructors information for Universal Connection Pool Manager MBean
UCP-45363	Runtime exception occurred while building attributes information for Universal Connection Pool Manager MBean
UCP-45364	Runtime exception occurred while building operations information for Universal Connection Pool Manager MBean
UCP-45365	Universal Connection Pool must be an instance of ConnectionConnectionPool or OracleConnectionConnectionPool
UCP-45366	Invalid MBean Descriptor is specified. Check the JDBC Universal Connection Pool MBean Descriptor
UCP-45367	Runtime exception occurred while building MBeanInfo for JDBC Universal Connection Pool MBean
UCP-45368	Runtime exception occurred while building constructors information for JDBC Universal Connection Pool MBean
UCP-45369	Runtime exception occurred while building attributes information for JDBC Universal Connection Pool MBean
UCP-45370	Runtime exception occurred while building operations information for JDBC Universal Connection Pool MBean
UCP-45371	Runtime exception occurred while building attributes information for Universal Connection Pool MBean
UCP-45372	Runtime exception occurred while building operations information for Universal Connection Pool MBean
UCP-45373	Invalid MBean Descriptor is specified. Check the Universal Connection Pool MBean Descriptor
UCP-45374	Runtime exception occurred while building MBeanInfo for Universal Connection Pool MBean
UCP-45375	Cannot stop the UCP metric collection. Exception occurred while trying to stop the metric collection or while destroying the nouns or sensors.
UCP-45376	Metrics update timer task scheduling failed
UCP-45377	Problem occurred while updating UCP metric sensors
UCP-45378	Universal Connection Pool is not an instance of OracleJDBCConnectionPool and cannot access ONSConfiguration property
UCP-45379	Cannot set the connection pool name in Universal Connection Pool MBean. Check the connection pool name to avoid duplicates
UCP-45380	MBean object is null
UCP-45381	MBean object name is null
UCP-45382	MBean display name is null
UCP-45383	Invalid adapter for pool creation in Universal Connection Pool Manager
UCP-45384	Invalid adapter for pool creation in Universal Connection Pool Manager MBean

Table A-1 (Cont.) Connection Pool Layer Error Messages

Error Message Number	Message
UCP-45385	Error during pool creation in Universal Connection Pool Manager
UCP-45386	Error during pool creation in Universal Connection Pool Manager MBean
UCP-45401	Waiting threads LO watermark cannot be negative
UCP-45402	Waiting threads HI watermark cannot be negative
UCP-45403	Total worker threads limit cannot be negative
UCP-45404	Queue poll timeout cannot be negative
UCP-45405	The waiting threads HI watermark cannot be lower than the LO watermark
UCP-45406	The limit of total worker threads cannot be higher than the limit of waiting threads
UCP-45407	The error number is out of range
UCP-45408	Invalid operation because the logger is null

A.3 JDBC Data Sources and Dynamic Proxies Error Messages

This section lists UCP error messages for JDBC data sources and dynamic proxies error messages.

Table A-2 JDBC Data Sources and Dynamic Proxies Error Messages

Error Message Number	Message
SQL-0	Unable to start the Universal Connection Pool
SQL-1	Unable to build the Universal Connection Pool
SQL-2	Invalid minimum pool size
SQL-3	Invalid maximum pool size
SQL-4	Invalid inactive connection timeout
SQL-5	Invalid connection wait timeout
SQL-6	Invalid time-to-live connection timeout
SQL-7	Invalid abandoned connection timeout
SQL-8	Invalid timeout check interval
SQL-9	Failed to enable Failover
SQL-10	Failed to set the maxStatements value
SQL-11	Failed to set the SQL string for validation
SQL-12	Invalid connection harvest trigger count
SQL-13	Invalid connection harvest max count
SQL-14	Universal Connection Pool is created already. Can not create the Universal Connection Pool again
SQL-15	Exception occurred while destroying the Universal Connection Pool
SQL-16	Operation only applies to Oracle connection pools
SQL-17	Exception occurred while setting ONS configuration string

Table A-2 (Cont.) JDBC Data Sources and Dynamic Proxies Error Messages

Error Message Number	Message
SQL-18	Failed to register labeling callback
SQL-19	Failed to remove labeling callback
SQL-20	Failed to register affinity callback
SQL-21	Failed to remove affinity callback
SQL-22	Invalid Universal Connection Pool configuration
SQL-23	Unable to create factory class instance with provided factory class name
SQL-24	Unable to set the User
SQL-25	Unable to set the Password
SQL-26	Unable to set the URL
SQL-27	The factory class must be an instance of DataSource
SQL-28	Cannot create connections. There are no available connections
SQL-29	Exception occurred while getting connection
SQL-30	Universal Connection Pool is not started
SQL-31	The connection is closed
SQL-32	Error occurred when applying label
SQL-33	Error occurred when removing the connection label
SQL-34	Error occurred when getting labels
SQL-35	Error occurred when getting unmatched labels
SQL-36	Error occurred when setting connection harvestable
SQL-37	Error occurred when registering harvesting callback
SQL-38	Error occurred when removing harvesting callback
SQL-39	Error occurred when registering abandoned-connection callback
SQL-40	Error occurred when removing abandoned-connection callback
SQL-41	Error occurred when registering time-to-live-connection callback
SQL-42	Error occurred when removing time-to-live-connection callback
SQL-43	The result set is closed
SQL-44	The statement is closed
SQL-45	Cannot set the connection pool name. Check the connection pool name to avoid duplicates
SQL-46	The SQL string is null
SQL-47	Error occurred when setting connection to be invalid
SQL-48	Unable to set the Connection properties
SQL-49	Unable to set the Database server name
SQL-50	Unable to set the Database port number
SQL-51	Unable to set the Database name
SQL-52	Unable to set the data source name
SQL-53	Unable to set the data source description
SQL-54	Unable to set the data source network protocol

Table A-2 (Cont.) JDBC Data Sources and Dynamic Proxies Error Messages

Error Message Number	Message
SQL-55	Unable to set the data source role name
SQL-56	Invalid max connection reuse time
SQL-57	Invalid max connection reuse count
SQL-58	The method is disabled
SQL-59	Unable to set the connection factory properties

Index

A

- abandon connection timeout property, [5](#)
- AbandonedConnectionTimeoutCallback, [1](#)
- admin package, [3](#)
- affinity
 - transaction-based, [15](#)
 - web session, [15](#)
- API overview, [3](#)
- application continuity
 - connection initialization callback, [2](#)
 - connection labeling, [2](#)
 - data source configuration, [1](#)
- Application Continuity, [1](#)
- applyConnectionLabel, [5](#)
- applying connection labels, [5](#)

B

- basic connection example, [3](#)
- benefits of connection pools, [1](#)
- benefits of FCF, [3](#)
- benefits of run-time connection load balancing, [13](#)
- borrowing connections
 - basic steps, [1](#)
 - conceptual architecture, [2](#)
 - labeled, [6](#)
 - overview, [1](#)
 - using JNDI, [4](#)
 - using the pool-enabled data source, [2](#)
 - using the pool-enabled XA data source, [3](#)

C

- caching statements, [10](#)
- callback
 - connection affinity, [17](#)
 - labeling, [2](#)
- checking unmatched labels, [6](#)
- closing connections, [9](#)
- conceptual architecture, [2](#)
- configure method, [2](#)
- Configuring ONS, [6](#)
 - client-side daemon configuration, [10](#)
 - Remote Configuration, [9](#)

- connection affinity
 - create callback, [17](#)
 - overview, [15](#)
 - register callback, [18](#)
 - remove callback, [18](#)
 - setting up, [17](#)
 - statistics, [3](#)
 - transaction-based, [15](#)
 - web session, [15](#)
- connection factory, [2](#)
 - conceptual architecture, [2](#)
 - requirements, [1](#)
 - setting, [2](#), [3](#)
- connection labels
 - apply, [5](#)
 - check unmatched, [6](#)
 - implement callback, [2](#)
 - overview, [1](#)
 - removing, [6](#)
- Connection object, [2](#)
- connection pool
 - benefits, [1](#)
 - create explicitly, [2](#)
 - create implicitly, [1](#), [1](#)
 - destroy, [3](#)
 - general overview, [1](#)
 - maintenance, [3](#)
 - purge, [4](#)
 - recycle, [4](#)
 - refresh, [4](#)
 - remove connection from, [10](#)
 - start, [3](#)
 - stop, [3](#)
 - understanding lifecycle, [1](#)
- connection pool manager,
 - create, [1](#)
 - create pool explicitly, [2](#)
 - destroy pool, [3](#)
 - overview, [3](#), [1](#)
 - purge pool, [4](#)
 - recycle pool, [4](#)
 - refresh pool, [4](#)
 - start pool, [3](#)
 - stop pool, [3](#)
- connection pool properties,
 - abandon connection timeout, [5](#)

connection pool properties (*continued*)

- connection wait timeout, [6](#)
- harvest maximum count, [9](#)
- harvest trigger count, [9](#)
- inactive connection timeout, [7](#)
- initial pool size, [2](#)
- maximum connection reuse count, [5](#)
- maximum connection reuse time, [4](#)
- maximum pool size, [2](#)
- maximum statements, [10](#)
- minimum pool size, [2](#)
- optimizing, [1](#)
- overview, [3](#)
- setting, [6](#), [1](#)
- time-to-live connection timeout, [6](#)
- timeout check interval, [7](#)
- validate on borrow, [7](#)

connection properties, [4](#)

connection reuse properties, setting, [4](#)

connection steps, basic, [1](#)

- example, [3](#)

connection URL, [12](#)

connection wait timeout property, [6](#)

ConnectionAffinityCallback interface, [17](#)

ConnectionLabelingCallback interface, [1](#), [2](#)

connections

- basic steps, [1](#)
- borrowing, [1](#)
- borrowing labeled, [6](#)
- borrowing using JNDI, [4](#)
- checking if valid, [8](#)
- closing, [9](#)
- controlling stale, [4](#)
- harvesting, [8](#)
- labeling, [1](#)
- removing from the pool, [10](#)
- run-time load balancing, [13](#)
- using affinity, [15](#)
- validate on borrow, [7](#)

cost method, [2](#)

create connection pool

- explicit, [2](#)
- implicit, [1](#)

D

data source

- PoolDataSource, [2](#), [2](#)
- PoolXADataSource, [2](#), [3](#)

database requirements, [1](#)

destroyConnectionPool, [3](#)

destroying a connection pool, [3](#)

E

enable FCF property, [6](#)

errors

- connection pool layer messages, [A-1](#)
- general UCP message structure, [A-1](#)
- JDBC data sources and dynamic proxies messages, [A-5](#)

example

- basic connection, [3](#)
- connection affinity callback, [17](#)
- FCF, [5](#)
- labeling callback, [3](#)

F

fast connection failover

- prerequisites, [5](#)

Fast Connection Failover

- See FCF

FCF,

- configure connection URL, [12](#)
- configure ONS, [6](#)
- enable, [6](#)
- example, [5](#)
- statistics, [2](#)

G

GDS, [19](#)

getAffinityPolicy, [17](#)

getConnection methods, [2](#), [6](#)

getPoolDataSource, [2](#)

getPoolXADataSource, [3](#)

getStatistics, [2](#)

getting a connection, [2](#)

getting an XA connection, [3](#)

getUniversalConnectionPoolManager, [1](#)

getUnmatchedConnectionLabels, [6](#)

getXAConnection methods, [3](#)

Global Data Services, [19](#)

H

harvest connections, [8](#)

harvest maximum count property, [9](#)

harvest trigger count property, [9](#)

HarvestableConnection interface, [8](#)

high availability, [3](#), [1](#)

I

inactive connection timeout property, [7](#)

initial pool size property, [2](#)

integration

- third-party, [10](#)

isValid, [8](#)

J

JDBC connection pool

See UCP

JDBC driver

connection properties, [4](#)

requirements, [1](#)

jdbc package, [3](#)

JNDI, [4](#)

JRE requirements, [1](#)

L

LabelableConnection interface, [1](#), [5](#)

labeled connections

apply label, [5](#)

borrowing, [6](#)

check unmatched, [6](#)

implement callback, [2](#)

overview, [1](#)

remove label, [6](#)

labeling callback

create, [2](#)

example, [3](#)

register, [4](#)

removing, [4](#)

run-time algorithm, [2](#)

lifecycle of connection pools, [1](#)

lifecycle states, [2](#)

Load Balance Advisory, [13](#)

load balancing, [12](#), [13](#)

logging, [3](#)

logging configuration

programmatically, [4](#)

properties file, [3](#)

logging levels, [5](#)

M

manager, connection pool, [1](#)

maximum connection reuse count property, [5](#)

maximum connection reuse time property, [4](#)

maximum pool size property, [2](#)

maximum statements property, [10](#)

method, [3](#)

minimum pool size property, [2](#)

O

ONS, [6](#)

ons.config file, [6](#)

optimizing a connection pool, [1](#)

Oracle Client software, [6](#)

Oracle Client software requirements, [1](#)

Oracle Notification Service

See ONS

Oracle RAC

connection affinity, [15](#)

features overview, [1](#)

run-time connection load balancing, [13](#)

statistics, [2](#)

Oracle RAC Load Balance Advisory, [13](#)

overview

API, [3](#)

connection pool manager, [1](#)

connection pool properties, [1](#)

connection pools, general, [1](#)

connection steps, [1](#)

high availability and performance features, [3](#)

labeling connections, [1](#)

Oracle RAC features, [1](#)

UCP, [2](#)

P

password, [2](#), [2](#), [3](#)

pool manager

See connection pool manager

pool properties

See connection pool properties

pool size, controlling

initial size, [2](#)

maximum, [2](#)

minimum, [2](#)

pool-enabled data source

create instance, [2](#)

pool-enabled XA data source

create instance, [3](#)

PoolDataSource interface, [2](#), [2](#)

PoolDataSourceFactory class, [2](#), [3](#)

PoolDataSourceImpl, [10](#)

PoolXADataSource interface, [2](#), [3](#)

PoolXADataSourceImpl, [10](#)

purgeConnectionPool, [4](#)

purging a connection pool, [4](#)

R

Real Application Clusters

See Oracle RAC, [2](#)

recycleConnectionPool, [4](#)

recycling a connection pool, [4](#)

refreshConnectionPool, [4](#)

refreshing a connection pool, [4](#)

registerConnectionAffinityCallback, [18](#)

registerConnectionLabelingCallback, [4](#)

removeConnectionAffinityCallback, [18](#)

removeConnectionLabel, [6](#)

removeConnectionLabelingCallback, [4](#)

removing connection labels, [6](#)

removing connections from the pool, [10](#)

reuse properites
 maximum count, [5](#)
 reuse properties
 maximum time, [4](#)
 run-time connection load balancing
 overview, [13](#)
 setting up, [14](#)
 statistics, [3](#)

S

SERVICE_TIME, [14](#)
 setAbandonedConnectionTimeout, [5](#)
 setAffinityPolicy, [17](#)
 setConnectionAffinityContext, [17](#)
 setConnectionFactoryClassName, [2](#), [3](#)
 setConnectionHarvestable, [8](#)
 setConnectionHarvestMaxCount, [9](#)
 setConnectionHarvestTriggerCount, [9](#)
 setConnectionProperties, [4](#)
 setConnectionWaitTimeout, [6](#)
 setFastConnectionFailoverEnabled, [6](#)
 setInactiveConnectionTimeout, [7](#)
 setInitialPoolSize, [2](#)
 setInvalid, [8](#), [10](#)
 setMaxConnectionReuseCount, [5](#)
 setMaxConnectionReuseTime, [4](#)
 setMaxPoolSize, [2](#)
 setMaxStatements, [10](#)
 setMinPoolSize, [2](#)
 setONSConfiguration, [6](#)
 setPassword, [2](#), [3](#)
 setSQLForValidateConnection, [7](#)
 setTimeoutCheckInterval, [7](#)
 setTimeToLiveConnectionTimeout, [6](#)
 setURL, [2](#), [3](#)
 setUser, [2](#), [3](#)
 setValidateConnectionOnBorrow, [7](#)
 SHORT, [14](#)
 SQL statement caching, [10](#)
 stale connections, [4](#)
 startConnectionPool, [3](#)
 starting a connection pool, [3](#)
 statement caching, [10](#)
 statistics
 connection affinity, [3](#)
 FCF, [2](#)
 Oracle RAC, [2](#)
 run-time connection load balancing, [3](#)
 stopConnectionPool, [3](#)
 stopping a connection pool, [3](#)

T

third-party integration, [10](#)
 THROUGHPUT, [14](#)
 time-to-live connection timeout property, [6](#)
 timeout check interval property, [7](#)
 timeout properties
 abandon, [5](#)
 check interval, [7](#)
 inactive, [7](#)
 time-to-live, [6](#)
 wait, [6](#)
 TimeToLiveConnectionTimeoutCallback, [1](#)
 transaction-based affinity, [15](#)

U

UCP,
 API overview, [3](#)
 basic connection steps, [1](#)
 conceptual architecture, [2](#)
 Oracle RAC features, [1](#)
 overview, [2](#)
 software requirements, [1](#)
 UCP for JDBC
 connection pool properties, [6](#), [1](#)
 UCP manager
 See connection pool manager
 ucp package, [3](#)
 universal connection pool
 See UCP
 UniversalConnectionPoolManager interface, [1](#)
 UniversalConnectionPoolManagerImpl, [1](#)
 unmatched labels, [6](#)
 URL, [2](#), [2](#), [3](#), [12](#)
 username, [2](#), [2](#), [3](#)

V

validate connections
 on borrow, [7](#)
 programmatically, [8](#)
 ValidConnection interface, [8](#), [10](#)

W

web session affinity, [15](#)

X

XA connections, [2](#), [3](#)
 XAConnection object, [2](#)